КЫРГЫЗСКО-РОССИЙСКИЙ СЛАВЯНСКИЙ УНИВЕРСИТЕТ

ЕСТЕСТВЕННО-ТЕХНИЧЕСКИЙ ФАКУЛЬТЕТ

Кафедра информационных и вычислительных технологий

В.И. Новоселов

МАТЕМАТИЧЕСКАЯ ЛОГИКА И ТЕОРИЯ АЛГОРИТМОВ

Учебно-методическое пособие

Бишкек 2011

УДК 519.712:681.142

Рецензенты:
доктор физ.-мат. наук *С.С. Искандеров*,
кандидат технических наук *М.С. Осмонов*

Рекомендовано к изданию
кафедрой информационных и вычислительных технологий

Новоселов В.И.

Н 76 МАТЕМАТИЧЕСКАЯ ЛОГИКА И ТЕОРИЯ АЛГОРИТМОВ: Учебно-методическое пособие. Бишкек: Изд-во КРСУ, 2011. 69 с.

Приведены и рассмотрены основные вопросы математической логики и теории алгоритмов, соответствующие курсу технических университетов. Ко всем разделам приведены задачи с указаниями и решениями.

Предназначено для студентов специальности «Программное обеспечение вычислительной техники и автоматизированных систем» в рамках образовательного направления 654600 «Информатика и вычислительная техника».

© КРСУ, 2011

СОДЕРЖАНИЕ

ВВЕДЕНИЕ....4
Глава 1. ВЫСКАЗЫВАНИЯ, ФОРМУЛЫ, ТАВТОЛОГИИ....5
Отношения логической эквивалентности
и логического следствия....8
Задачи....9
Глава 2. ФОРМАЛЬНЫЕ ТЕОРИИ....12
Глава 3. ИСЧИСЛЕНИЕ ВЫСКАЗЫВАНИЙ....13
Построение вывода в логике высказываний....17
Задачи....19
Глава 4. МЕТОД РЕЗОЛЮЦИЙ В ЛОГИКЕ ВЫСКАЗЫВАНИЙ....20
Задачи....24
Глава 5. ПРЕДИКАТЫ....24
Задачи....31
Глава 6. ИСЧИСЛЕНИЕ ПРЕДИКАТОВ....33
Теория равенства....38
Формальная арифметика....39
Теория частично упорядоченных множеств....39
Задачи....40
Глава 7. АЛГОРИТМЫ....42
Глава 8. РЕКУРСИВНЫЕ ФУНКЦИИ....43
Задачи....49
Глава 9. МАШИНЫ ТЬЮРИНГА....50
Операции с машинами Тьюринга....54
Принцип двойственности....54
Способы композиции машин Тьюринга....55
Задачи....56
Ответы и указания....61
Глава 10. ЭФФЕКТИВНОСТЬ АЛГОРИТМОВ....63
Переборные задачи и сложность вычислений....63
Классы задач P и NP....63
Недетерминированные вычисления....63
Класс NP-полных задач....65
Труднорешаемые задачи....65
NP-трудные задачи....66
ЛИТЕРАТУРА....67

ВВЕДЕНИЕ

Логикой называют науку о законах и формах мышления. Формальная логика изучает формы правильных рассуждений. Математическая логика – часть формальной логики – изучает формы рассуждений, принятые в математике.

В настоящее время значение математической логики в инженерном образовании возрастает в связи с необходимостью развития математического мышления, а также изучения дискретной или компьютерной математики. Математическая логика и теория алгоритмов являются продолжением курса дискретной математики, что нашло отражение в данном учебном пособии. Учебное пособие адресовано студентам, обучающимся по специальности 230105 – «Программное обеспечение вычислительной техники и автоматизированных систем». Теоретическая часть пособия основана на лекциях, читанных автором в КРСУ.

Сейчас издается достаточно большое количество учебной литературы по дискретной математике. В этой литературе (в особенности, [3], [12], [14]) уделяется внимание и математической логике, и теории алгоритмов. Тем не менее, и указанные пособия, и задачники, в частности, [2] и [6], ориентированы все же на математиков, а не на студентов инженерных специальностей (исключениями являются, пожалуй, только [1], [8], [13], на идеях которых основывался автор). Поэтому в настоящем пособии приведено достаточно много задач, причем решения многих задач подробно разобраны.

Предполагается, что студент, начинающий работу с данным учебным пособием, знаком с основными понятиями теории множеств и отношений, а также с аппаратом булевых функций.

Автор выражает благодарность своим рецензентам и своим коллегам по кафедре за содержательные консультации и внимание к работе.

Глава 1. ВЫСКАЗЫВАНИЯ, ФОРМУЛЫ, ТАВТОЛОГИИ

Определение. *Высказыванием* называется утверждение, которое является истинным или ложным (но не одновременно).

То есть, чтобы выяснить, является ли некоторое предложение высказыванием, нужно сначала убедиться, что это утверждение, а затем установить, истинно оно или ложно.

Пример. «Москва – столица России» – истинное высказывание.

«5 – четное число» – ложное высказывание.

«$x-1=4$» – не высказывание (неизвестно, какие значения принимает x).

«Студент второго курса» не высказывание (не является утверждением).

Высказывания бывают элементарные и составные.

Элементарные высказывания не могут быть выражены через другие высказывания. *Составные* высказывания можно выразить с помощью элементарных высказываний.

Пример. «Число 22 четное» – элементарное высказывание.

«Число 22 четное и делится на 11» – составное высказывание.

Высказывания обозначают заглавными буквами латинского алфавита: A, B, C,… Эти буквы называют логическими *атомами*.

При фиксированном множестве букв $A=\{A,B,C,...\}$ *интерпретацией* называется функция I, которая отображает множество A во множество истинностных (логических) значений $T=\{\text{истина, ложь}\}$, то есть $I: A \to T$.

Истинностные значения истина и ложь сокращенно обозначаются и, л или T, F, или 1,0. Мы будем использовать обозначения 1 и 0. В определенной интерпретации буквы принимают значения 1 или 0.

К высказываниям и буквам можно применять известные из курса дискретной математики логические связки или логические операции. При этом получаются *формулы (формы)*. Формулы становятся высказываниями при подстановке всех значений букв.

Таблица истинности основных логических операций.

A	B	$\neg A$	$A \wedge B$	$A \vee B$	$A \to B$	$A \leftrightarrow B$
0	0	1	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	1	0	0
1	1	0	1	1	1	1

Более строго формула определяется так.

Определение. 1) Всякая буква есть формула.

2) Если A, B – формулы, то формулами являются также $\neg A$, $A \vee B$, $A \wedge B$, $A \to B$, $A \leftrightarrow B$.

3) Символ является формулой тогда и только тогда, когда это следует из 1) и 2).

В классической логике формулы принято заключать в круглые скобки, но мы этого делать не будем. Для всякой формулы можно построить таблицу истинности.

Значение формулы F в заданной интерпретации I обозначают $|F|$ (или $|F|_I$, или $I(F)$).

Часть формулы, которая сама является формулой, называется *подформулой* данной формулы.

Определение. Формула называется *тавтологией*, если она принимает только истинные значения при любых значениях букв.

Другими словами, тавтология – это тождественно истинная формула.

Установить, является ли формула тавтологией, можно:

– по таблице истинности,

– используя свойства логических операций.

Из курса дискретной математики известны основные логические эквивалентности (свойства логических операций), которые являются примерами тавтологий.

1. Коммутативность: $A \vee B = B \vee A$, $A \wedge B = B \wedge A$.
2. Ассоциативность:
   $A \vee (B \vee C) = (A \vee B) \vee C$, $A \wedge (B \wedge C) = (A \wedge B) \wedge C$.
3. Дистрибутивность:
   $A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$, $A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$.
4. Идемпотентность: $A \vee A = A$, $A \wedge A = A$.
5. Закон двойного отрицания: $\neg\neg A = A$.
6. Закон исключения третьего: $A \vee \neg A = 1$.
7. Закон противоречия: $A \wedge \neg A = 0$.
8. Законы де Моргана:
   $\neg(A \vee B) = \neg A \wedge \neg B$, $\neg(A \wedge B) = \neg A \vee \neg B$.
9. Свойства операций с логическими константами:
   $A \vee 1 = 1$, $A \vee 0 = A$, $A \wedge 1 = A$, $A \wedge 0 = 0$.

Здесь A, B и C – любые буквы.

Примеры. 1. Доказать, что формула $A \wedge B \to A$ является тавтологией.

Доказательство. Допустим, что при некоторых значениях букв (то есть в некоторой интерпретации)

$$|A \wedge B \to A| = 0 \Leftrightarrow \begin{cases} |A \wedge B| = 1, \\ \\ |A| = 0 \end{cases} \Leftrightarrow \begin{cases} |A| = 1, \\ |B| = 1, \\ |A| = 0. \end{cases}$$

Приходим к противоречию, которое доказывает, что исходная формула – тавтология.

2. Доказать, что формула $(A \wedge B) \wedge C \leftrightarrow A \wedge (B \wedge C)$ является тавтологией.

Доказательство. Эквиваленция истинна, если левая и правая части принимают одинаковые значения на некотором наборе значений букв.

Допустим, что при некоторых значениях букв

$$|(A \wedge B) \wedge C| = 1 \Leftrightarrow \begin{cases} |A \wedge B| = 1, \\ \\ |C| = 1, \end{cases} \Leftrightarrow \begin{cases} |A| = 1, \\ |B| = 1, \\ |C| = 1, \end{cases} \Leftrightarrow \begin{cases} |A| = 1, \\ \\ |B \wedge C| = 1, \end{cases} \Leftrightarrow |A \wedge (B \wedge C)| = 1.$$

Следовательно, исходная формула – тавтология.

3. Доказать, что формула $(A \vee B) \vee C \leftrightarrow A \vee (B \vee C)$ является тавтологией.

Доказательство. Допустим, что при некоторых значениях букв

$$|(A \vee B) \vee C| = 0 \Leftrightarrow \begin{cases} |A \vee B| = 0, \\ \\ |C| = 0, \end{cases} \Leftrightarrow \begin{cases} |A| = 0, \\ |B| = 0, \\ |C| = 0, \end{cases} \Leftrightarrow \begin{cases} |A| = 0, \\ \\ |B \vee C| = 0, \end{cases} \Leftrightarrow |A \vee (B \vee C)| = 0.$$

Следовательно, исходная формула – тавтология.

Таким образом, тождественную истинность импликации удобно доказывать от противного, а тождественную истинность эквиваленции установлением равенства значений левой и правой части.

Теорема. Пусть формулы A и $A \to B$ – тавтологии. Тогда формула B – тавтология.

Доказательство. Пусть P_1, P_2, …, P_k – буквы в формулах A и $A \to B$. В теории булевых функций было доказано, что все булевы функции, а, следовательно, и формулы, можно считать зависящими от одного и того же количества букв. Рассмотрим некоторый набор значений σ_1, σ_2, …, σ_k, где $\sigma_i \in \{0,1\}$, $i = 1,2,...,k$. Подставим данный набор значений в формулы A и $A \to B$ вместо соответствующих букв. Формулы являются тавтологиями по условию теоремы, следовательно, $A(\sigma_1,\sigma_2,...,\sigma_k)=1$ и $A(\sigma_1,\sigma_2,...,\sigma_k) \to B(\sigma_1,\sigma_2,...,\sigma_k)=1$.

По таблице истинности импликации получаем, что $B(\sigma_1, \sigma_2, ..., \sigma_k) = 1$. Поскольку набор значений σ_1, σ_2, …, σ_k был произволен, формула B – тавтология, что и требовалось доказать.

Теорема. Пусть формула A – тавтология, P_1, P_2, …, P_k – буквы в формуле A, A_1, A_2, …, A_k – любые формулы. Тогда новая формула $B = A(A_1, A_2, ..., A_k)$ – тавтология.

Доказательство аналогично доказательству предыдущей теоремы.

Отношения логической эквивалентности и логического следствия

Определение. Формулы A и B называются логически эквивалентными тогда и только тогда, когда формула $A \leftrightarrow B$ – тавтология.

Замечание. Формула $A \leftrightarrow B$ – тавтология, если таблицы истинности формул A и B совпадают.

Пример. По законам де Моргана, логически эквивалентны формулы $\neg(A \wedge B)$ и $\neg A \vee \neg B$, а также формулы $\neg(A \vee B)$ и $\neg A \wedge \neg B$.

Теорема. Отношение логической эквивалентности является отношением эквивалентности.

Рефлексивность, симметричность и транзитивность данного отношения следуют из замечания.

Справедливы правило подстановки и правило замены.

Пусть F и G – формулы, содержащие букву A, F_H^A и G_H^A – формулы, полученные из формул F и G соответственно подстановкой вместо буквы A формулы H.

Правило подстановки. Если формула F логически эквивалентна формуле G, то формула F_H^A логически эквивалентна формуле G_H^A.

Пусть F_G – формула, в которой выделена некоторая подформула G, F_H – формула, полученная из формулы F_G заменой G на некоторую формулу H.

Правило замены. Если формулы G и H логически эквивалентны, то логически эквивалентны и формулы F_G и F_H.

Доказательства правил подстановки и замены основано на сравнении таблиц истинности соответствующих формул.

Пример. Известна тавтология $(A \to B) \leftrightarrow (\neg A \vee B)$ (проверьте самостоятельно). По правилу подстановки, формула $(\neg A \to B)$ логически эквивалентна формуле $(\neg\neg A \vee B)$. По правилу замены, примененному к закону двойного отрицания, получаем, что формула

$(\neg\neg A \vee B)$ логически эквивалентна формуле $(A \vee B)$. Следовательно, по свойству транзитивности, формулы $(\neg A \to B)$ и $(A \vee B)$ логически эквивалентны.

Определение. Говорят, что формула A логически влечет формулу B (из формулы A логически следует формула B), если формула $A \to B$ является тавтологией.

Теорема. Отношение логического следствия является отношением предпорядка, то есть рефлексивным и транзитивным отношением.

Пример. Формула $A \wedge B$ логически влечет формулу A. В самом деле, в примере 1 предыдущего пункта было доказано, что формула $A \wedge B \to A$ является тавтологией.

Задачи

1. Установить, является ли предложение высказыванием, и если является, истинно оно или ложно.

1) Волга впадает в Каспийское море.
2) Студент второго курса.
3) $x^2 - 5x + 6$.
4) $x^2 - 5x + 6 = 0$.
5) Существует человек, который не старше своего отца.
6) $2 < 4$.
7) Марс есть спутник Земли.
8) $\sqrt{3} + 16 - \sqrt[3]{7}$.
9) $2 + \sqrt{16} = 6$.
10) Который час?

2. Установить, является ли предложение высказыванием, и если является, истинно оно или ложно.

1) $2 \in \left\{x \middle| 2x^3 - 3x^2 + 1 = 0, x \in R\right\}$.
2) $1 \in \left\{x \middle| x^3 + x^2 - x - 1 = 0, x \in Z\right\}$.
3) $\left\{x \middle| 2x^3 - 3x^2 = 0, x \in R\right\}$.
4) $0 \in \left\{x \middle| 2x^3 - 3x^2 + 1 = 0, x \in R\right\}$.
5) $\{1\} \in P(N)$.
6) $\varnothing$.
7) $\{1\} \in N$.

8) $\varnothing \subset N$.

9) $\{-1,2\} \subset \left\{x \middle| x^3 + x^2 - x - 1 = 0, x \in Z\right\}$.

10) $\{-1,1\} = \left\{x \middle| x^3 + x^2 - x - 1 = 0, x \in Z\right\}$.

3. Среди следующих высказываний выделить элементарные и составные. В составных высказываниях обозначить элементарные высказывания буквами и записать с помощью логических символов.

1) Число 6 является делителем числа 36.
2) Число 225 делится нацело на 5.
3) Число 225 делится нацело на 5 и не делится на 10.
4) Если 81 делится нацело на 9, то 81 делится на 3.
5) 16 кратно 2.
6) 18 кратно 2 и 3.
7) $2 \geq 1$.
8) Число 39 имеет 2 простых делителя.
9) Двузначное число 19 простое.
10) Корнями уравнения $x^2 - 5x + 6 = 0$ являются числа 2 и 3.

4. Пусть A обозначает высказывание «Я увлекаюсь горным туризмом», а B обозначает высказывание «Я изучаю программирование». Дайте словесную формулировку следующих высказываний:

1) $\neg A$; 2) $\neg\neg B$; 3) AB; 4) $A \vee B$; 5) $A\neg B$; 6) $\neg AB$;
7) $\neg A \vee \neg B$; 8) $A \to B$; 9) $A \leftrightarrow B$; 10) $\neg A \vee B$.

5. Проверить, является ли формула тавтологией, без построения таблицы истинности.

1) $A \to A$.
2) $\neg A \to A$.
3) $A \wedge \neg A$.
4) $A \vee \neg A$.
5) $A \leftrightarrow \neg A$.
6) $A \leftrightarrow A$.
7) $(A \vee A) \to A$.
8) $(A \wedge A) \to A$.
9) $(A \vee A) \to (A \wedge A)$.
10) $\neg(A \leftrightarrow \neg A)$.

6. Доказать, что формула является тавтологией, без построения таблицы истинности. Во всех формулах выделить всевозможные подформулы.

1) $(A \to B) \to ((A \vee C) \to (B \vee C))$.
2) $((A \to B) \wedge (B \to C)) \to (A \to C)$.

3) $\big((A \wedge B) \leftrightarrow B\big) \leftrightarrow (B \to A)$.

4) $(A \to B) \leftrightarrow (\neg B \to \neg A)$.

5) $\big((A \to B) \to A\big) \to A$.

6) $\neg A \to (A \to B)$.

7) $(\neg A \to B) \vee \neg(A \wedge B)$.

8) $(A \to B) \to (\neg B \to \neg A)$.

9) $(A \to C) \to ((B \to C) \to (A \vee B \to C))$.

10) $(A \to B) \to ((A \wedge C) \to (B \wedge C))$.

7. Доказать, что формулы логически эквивалентны.

1) $A \wedge (B \vee C)$ и $(A \wedge B) \vee (A \wedge C)$.

2) $A \to B$ и $\neg B \to \neg A$.

3) $\neg\neg A$ и A.

4) $\neg(A \vee B)$ и $\neg A \wedge \neg B$.

5) $(A \wedge B) \wedge C$ и $A \wedge (B \wedge C)$.

6) $A \vee (B \wedge C)$ и $(A \vee B) \wedge (A \vee C)$.

7) $\neg(A \wedge B)$ и $\neg A \vee \neg B$.

8) $(A \vee B) \vee C$ и $A \vee (B \vee C)$.

9) $A \to B$ и $\neg A \vee B$.

10) $A \vee (A \wedge B)$ и A.

8. Доказать, что первая формула логически влечет вторую формулу.

1) $\neg(A \vee B)$; $\neg(A \wedge B)$.

2) A; $B \to A$.

3) $(A \to B)(B \to C)$; $A \to C$.

4) $A \to B$; $(A \wedge C) \to (B \wedge C)$.

5) $A \to C$; $(B \to C) \to (A \vee B \to C)$.

6) A; $\neg\neg A$.

7) $\neg A$; $A \to B$.

8) $A \to B$; $(A \vee C) \to (B \vee C)$.

9) $A \to B$; $\neg B \to \neg A$.

10) $A \to (B \to C)$; $(A \to B) \to (A \to C)$.

9. Доказать теорему о том, что отношение логической эквивалентности является отношением эквивалентности.

10. Доказать теорему о том, что отношение логического следования является отношением предпорядка.

Глава 2. ФОРМАЛЬНЫЕ ТЕОРИИ

Одним из основных понятий математической логики является понятие формальной теории или исчисления. Это понятие было первоначально разработано для формализации логики и теории доказательств. Формальная теория является эффективным механизмом получения новых теорем. Кроме того, аппарат формальной теории позволяет решать задачи, связанные с математическим моделированием различных явлений и процессов.

Формальная теория считается заданной, если известны следующие четыре составляющих:

1. *Алфавит* – конечное или счетное множество символов.
2. *Формулы*, которые по специальным правилам строятся из символов алфавита. Формулы, как правило, составляют счетное множество.

Алфавит и формулы определяют язык или сигнатуру формальной теории.

3. *Аксиомы* – выделенное из множества формул специальное подмножество. Множество аксиом может быть конечным или бесконечным. Бесконечное множество аксиом задается с помощью конечного множества *схем аксиом* и правил порождения конкретных аксиом из схемы аксиом. Различают два вида аксиом: *логические* (общие для класса формальных теорий) и *собственные* (определяющие содержание конкретной теории).
4. *Правила вывода* – множество отношений (как правило, конечное) на множестве формул, позволяющие из аксиом получать теоремы формальной теории.

Обратите внимание, что здесь аксиомы – это необязательно утверждения, не требующие доказательства.

Определение. *Выводом* формальной теории называется последовательность формул A_1, A_2, …, A_n, в которой все формулы – либо аксиомы, либо получаются из предыдущих по правилам вывода.

Говорят, что формула A *выводима* из множества формул Γ (обозначение: $\Gamma \vdash A$), если существует вывод A_1, A_2, …, A_n, где $A_n = A$, и есть три возможности:

- $A_i \in \Gamma$;
- A_i – аксиома;
- A_i получаются из предыдущих формул по правилам вывода.

Формулы из множества Γ называются *посылками* или *гипотезами* вывода.

Примеры выводов мы рассмотрим в определенных формальных теориях.

В частном случае, когда $\Gamma = \varnothing$, имеет место обозначение: $\vdash A$, и формула Γ называется *выводимой* в данной теории (или *теоремой* данной теории). Иногда значок $\vdash$ записывается так: $\vdash_K$, где K – обозначение данной теории.

Глава 3. ИСЧИСЛЕНИЕ ВЫСКАЗЫВАНИЙ

Исчисление высказываний (теория *L*) определяется следующими компонентами.

1. Алфавит составляют:

- Пропозициональные буквы (от англ. proposition – высказывание) – заглавные буквы латинского алфавита (иногда с индексами – натуральными числами): $A, B, C, \ldots, A_1, B_1, C_1, \ldots$
- Логические связки: $\neg, \rightarrow$.
- Скобки: (,).

Иногда в исчислении высказываний допускаются формулы с другими логическими связками, но при этом учитывается, как они выражаются через инверсию и импликацию. Так, $A \wedge B = \neg(A \rightarrow \neg B)$, $A \vee B = \neg A \rightarrow B$. Такие записи являются удобными сокращениями.

2. Формулы определяются так же, как в главе 1.

Определение. 1) Всякая пропозициональная буква есть формула.

2) Если A, B – формулы, то формулами являются также $\neg A$, $A \rightarrow B$.

3) Символ является формулой тогда и только тогда, когда это следует из 1) и 2).

3. Аксиомы задаются тремя схемами аксиом:

A1. $A \rightarrow (B \rightarrow A)$.

A2. $(A \rightarrow (B \rightarrow C)) \rightarrow ((A \rightarrow B) \rightarrow (A \rightarrow C))$.

A3. $(\neg B \rightarrow \neg A) \rightarrow ((\neg B \rightarrow A) \rightarrow B)$.

Существуют исчисления высказываний с другим набором логических связок и другими схемами аксиом, но в данном пособии они не рассматриваются. Желающие могут ознакомиться с ними в [12].

4. Правило вывода Modus ponens (сокращенно MP) – правило отделения (лат.).

$A, A \rightarrow B \vdash B$.

Здесь A, B – любые формулы. Таким образом, множество аксиом исчисления высказываний, заданное тремя схемами аксиом, бесконечно. Множество правил вывода задано одной схемой, и также бесконечно.

Теорема. Все теоремы исчисления высказываний – тавтологии.

Доказательство. Докажем сначала, что аксиомы А1 – А3 являются тавтологиями.

Предположим, что

$$|A \to (B \to A)| = 0 \Leftrightarrow \begin{cases} |A| = 1, \\ |B \to A| = 0, \end{cases} \Leftrightarrow \begin{cases} |A| = 1, \\ |B| = 1, \\ |A| = 0. \end{cases}$$

Полученное противоречие доказывает, что аксиома А1 – тавтология.

Предположим, что

$$|(A \to (B \to C)) \to ((A \to B) \to (A \to C))| = 0 \Leftrightarrow$$

$$\Leftrightarrow \begin{cases} |A \to (B \to C)| = 1, \\ |(A \to B) \to (A \to C)| = 0, \end{cases} \Leftrightarrow \begin{cases} |A \to (B \to C)| = 1, \\ |A \to B| = 1, \\ |A \to C| = 0, \end{cases} \Leftrightarrow$$

$$\Leftrightarrow \begin{cases} |A \to (B \to C)| = 1, \\ |A \to B| = 1, \\ |A| = 1, \\ |C| = 0, \end{cases} \Leftrightarrow \begin{cases} |A \to (B \to C)| = 1, \\ |B| = 1, \\ |A| = 1, \\ |C| = 0, \end{cases} \Leftrightarrow \begin{cases} |B \to C| = 1, \\ |B| = 1, \\ |A| = 1, \\ |C| = 0, \end{cases} \Leftrightarrow \begin{cases} |C| = 1, \\ |B| = 1, \\ |A| = 1, \\ |C| = 0. \end{cases}$$

Полученное противоречие доказывает, что аксиома А2 – тавтология.

Предположим, что

$$|(\neg B \to \neg A) \to ((\neg B \to A) \to B)| = 0 \Leftrightarrow \begin{cases} |\neg B \to \neg A| = 1, \\ |(\neg B \to A) \to B| = 0, \end{cases} \Leftrightarrow$$

$$\Leftrightarrow \begin{cases} |\neg B \to \neg A| = 1, \\ |\neg B \to A| = 1, \\ |B| = 0, \end{cases} \Leftrightarrow \begin{cases} |\neg B| = 1, \\ |\neg B \to \neg A| = 1, \\ |\neg B \to A| = 1, \\ |B| = 0, \end{cases} \Leftrightarrow \begin{cases} |\neg A| = 1, \\ |A| = 1, \\ |B| = 0, \end{cases} \Leftrightarrow \begin{cases} |A| = 0, \\ |A| = 1, \\ |B| = 0. \end{cases}$$

Полученное противоречие доказывает, что аксиома А3 – тавтология.

Таким образом, все аксиомы исчисления высказываний представляют собой тавтологии. Теоремы выводятся по правилу вывода MP, следовательно, по ранее полученным результатам (см. **Глава 1. ВЫСКАЗЫВАНИЯ, ФОРМУЛЫ, ТАВТОЛОГИИ**), также являются тавтологиями, что и требовалось доказать.

Следствие. Исчисление высказываний непротиворечиво.

Доказательство. Предположим противное, то есть в исчислении есть теоремы A и $\neg A$. По доказанной теореме, A и $\neg A$ являются тавтологиями (тождественно истинными формулами), следовательно, формула A одновременно является тождественно истинной и тождественно ложной, что является противоречием.

Лемма. $\vdash A \to A$.

Доказательство. Построим вывод формулы $A \to A$.

1. $A \to (A \to A)$. A1 с подстановкой вместо B – A.
2. $A \to ((A \to A) \to A)$. A1 с подстановкой вместо B – $A \to A$.
3. $(A \to ((A \to A) \to A)) \to ((A \to (A \to A)) \to (A \to A))$

A2 с подстановкой вместо C – A, а вместо B – $A \to A$.

4. $((A \to A) \to A) \to (A \to A)$. MP 2,3.
5. $A \to A$. MP 1,4.

Что и требовалось доказать.

Теорема дедукции. Пусть Γ – множество формул, A, B – формулы. Тогда Γ, $A \vdash B \Rightarrow \Gamma \vdash A \to B$.

В частности, если $\Gamma = \varnothing$, то если $A \vdash B \Rightarrow \vdash A \to B$.

Доказательство. Пусть B_1, B_2, …, $B_n = B$, – вывод из Γ и A. Методом математической индукции докажем, что $\Gamma \vdash A \to B_i$, $i = 1,2,...,n$.

1) Проверим, что утверждение $\Gamma \vdash A \to B_i$ справедливо при $i = 1$, то есть $\Gamma \vdash A \to B_1$.

Для B_1 возможны три варианта: $B_1 \in \Gamma$, B_1 – аксиома, $B_1 = A$.

а) Пусть $B_1 \in \Gamma$ или B_1 – аксиома. Построим вывод:

1. B_1.
2. $B_1 \to (A \to B_1)$. A1 с подстановкой вместо A – B_1, вместо B – A.
3. $A \to B_1$. MP 1, 2.

Таким образом, $\Gamma \vdash A \to B_1$.

б) Пусть $B_1 = A$. По лемме, $\vdash A \to A = A \to B_1$. Таким образом, $\Gamma \vdash A \to B_1$.

2) Пусть утверждение $\Gamma \vdash A \to B_i$ верно при $i = 1,2,...,k$, $k \le n$. Докажем утверждение для $i = k+1$, то есть $\Gamma \vdash A \to B_{k+1}$.

Для формулы B_{k+1} есть следующие возможности: $B_{k+1} \in \Gamma$, B_{k+1} – аксиома, $B_{k+1} = A$, которые рассматриваются аналогично предыдущему пункту, и новая возможность: B_{k+1} получается из предыдущих формул B_1, B_2, …, B_k, по правилу Modus ponens. Последний случай рассмотрим подробно.

Среди формул B_1, B_2, …, B_k есть формулы (может быть, и не одна) вида B_j, $1 \le j \le k$, такие, что имеет место формула $B_j \to B_{k+1}$ (которая также присутствует в выводе), поэтому и возможно применение правила Modus ponens.

По предположению индукции, $\Gamma \vdash A \to B_j$, $\Gamma \vdash A \to (B_j \to B_{k+1})$.

Построим вывод:

1. $A \to B_j$.
2. $A \to (B_j \to B_{k+1})$.
3. $(A \to (B_j \to B_{k+1})) \to ((A \to B_j) \to (A \to B_{k+1}))$. A2 с подстановкой вместо B – B_j, вместо C – B_{k+1}.
4. $(A \to B_j) \to (A \to B_{k+1})$. MP 2, 3.
5. $A \to B_{k+1}$.

Таким образом, доказано, что $\Gamma \vdash A \to B_{k+1}$, следовательно, по методу математической индукции, $\Gamma \vdash A \to B_n$, то есть $\Gamma \vdash A \to B$. Теорема доказана.

Справедлива и обратная теорема.

Теорема. $\Gamma \vdash A \to B \Rightarrow \Gamma, A \vdash B$.

Доказательство. Построим вывод:

1. Γ.
2. A.
3. $A \to B$. По условию теоремы, эта формула выводима из Γ.
4. B. MP 2, 3.

Теорема доказана.

На основании теоремы дедукции получена теорема о полноте исчисления высказываний. Доказательство этой теоремы довольно громоздко, поэтому желающие могут ознакомиться с ним в [12].

Теорема о полноте. Всякая тавтология является теоремой исчисления высказываний.

Следствие. Множество всех теорем исчисления высказываний совпадает с множеством всех тавтологий.

Теорема дедукции позволяет строить выводы многих формул в исчислении высказываний.

Построение вывода в логике высказываний

Пример. Докажем, что выводима формула $(\neg B \to \neg A) \to (A \to B)$. Сокращенно это записывается так: $\vdash (\neg B \to \neg A) \to (A \to B)$.

По теореме, обратной теореме дедукции, посылку можно перенести в левую часть:

$\neg B \to \neg A \vdash A \to B$.

Проделаем эту операцию еще раз:

$\neg B \to \neg A,\ A \vdash B$.

Таким образом, нам нужно доказать, что из формул $\neg B \to \neg A$ и A выводима формула B. Составим вывод формулы B. В каждой строке вывода записывается только одна формула. В правой части страницы удобно указывать комментарий, – что собой эта формула представляет. Возможны варианты:

- гипотеза,
- аксиома (может быть, с какими-то подстановками),
- ранее доказанная теорема,
- формула получена из предыдущих формул по правилу Modus ponens.

Вначале мы запишем гипотезы.

1. $\neg B \to \neg A$ – гипотеза.
2. A – гипотеза.

Формулу B удобно получить из аксиомы А3. Поэтому запишем эту аксиому:

3. $(\neg B \to \neg A) \to ((\neg B \to A) \to B)$ А3.

К формулам 1 и 3 можно применить правило вывода Modus ponens (что мы и отметим в комментарии). Порядок номеров формул существенен (первой указывается посылка).

4. $(\neg B \to A) \to B$. MP 1, 3.

Посылку в формуле 4 можно получить из аксиомы А1, если заменить B на $\neg B$:

5. $A \to (\neg B \to A)$. А1 с подстановкой вместо B – $\neg B$.

Далее дважды применяем правило Modus ponens:

6. $\neg B \to A$. MP 2, 5.
7. B. MP 6, 4.

Вывод построен, и применением теоремы дедукции мы доказали выводимость первоначальной формулы.

Отметим, что вывод может быть не единственным, в частности, формулы могут быть записаны в другом порядке. Решение данной задачи может быть оформлено следующим образом:

$\vdash (\neg B \to \neg A) \to (A \to B)$.

По теореме, обратной теореме дедукции,

$\neg B \to \neg A \vdash A \to B$.

$\neg B \to \neg A,\ A \vdash B$.

1. $\neg B \to \neg A$ – гипотеза.
2. A – гипотеза.
3. $A \to (\neg B \to A)$. A1, $B : \neg B$.
4. $\neg B \to A$. MP 2, 3.
5. $(\neg B \to \neg A) \to ((\neg B \to A) \to B)$. A3.
6. $(\neg B \to A) \to B$. MP 1, 5.
7. B. MP 4, 6.

Пример. Данный пример более прост, но достаточно показателен. Обратите внимание, что здесь не используются ни аксиомы, ни теоремы. Доказательство теоремы $\vdash (A \to B) \to ((C \to A) \to (C \to B))$ строится только на основании правила MP.

По теореме, обратной теореме дедукции,

$A \to B \vdash (C \to A) \to (C \to B)$.

$A \to B,\ C \to A \vdash C \to B$.

$A \to B,\ C \to A,\ C \vdash B$.

1. $A \to B$ – гипотеза.
2. $C \to A$ – гипотеза.
3. C – гипотеза.
4. A. MP 3,2.
5. B. MP 4,1.

Задачи

1. Указать, какие из следующих выражений являются формулами исчисления высказываний.

1) $A \to (B \to C)$.

2) $(A \to B) \to ((A \to C) \to (D \to C))$.

3) $x \to (A \to B) \wedge (B \to A)$.

4) $((A \to B) \wedge \to A) \to A$.

5) $(A \to B) \to ((A \to \neg B) \to A)$.

6) $\forall x A$.

7) $\neg\neg A \to A$.

8) $\neg A \to (A \vee B)$.

9) $(A \to B) \leftrightarrow (A \to B)$.

10) $\neg\neg A \to\to A$.

2. Какая из приведенных ниже записей является выводом формулы $(A \to (B \to C)) \to (B \to (A \to C))$ в исчислении высказываний?

1) По теореме, обратной теореме дедукции, $\vdash (A \to (B \to C)) \to (B \to (A \to C))$ равносильно $A \to (B \to C), B \vdash A \to C$.

   1. $A \to (B \to C)$ – гипотеза.
   2. B – гипотеза.
   3. $A \to C$. MP 2,1.

2) По теореме, обратной теореме дедукции, $\vdash (A \to (B \to C)) \to (B \to (A \to C))$ равносильно $A \to (B \to C), B, A \vdash C$.

   1. $A \to (B \to C)$ – гипотеза.
   2. B – гипотеза.
   3. A – гипотеза.
   4. $B \to C$. MP 3,1.
   5. C. MP 2,4.

3) По теореме, обратной теореме дедукции, $\vdash (A \to (B \to C)) \to (B \to (A \to C))$ равносильно $A \to (B \to C), B, A \vdash C$.

   1. $A \to (B \to C)$ – гипотеза.
   2. B – гипотеза.
   3. A – гипотеза.
   4. $B \to C$. MP 2,1.
   5. C. MP 2,4.

3. Построить вывод формулы.

1) $(A \to B) \to ((A \to (B \to C)) \to (A \to C))$.

2) $(A \to B) \to ((B \to C) \to (A \to C))$.

3) $(A \to B) \to ((B \to C) \to ((C \to D) \to (A \to D)))$.

4) $\neg A \to (A \to B)$.

5) $\neg\neg A \to A$.

6) $A \to \neg\neg A$.

7) $(A \to \neg\neg B) \to (A \to B)$.

8) $(\neg\neg A \to B) \to (A \to B)$.

9) $(A \to B) \to (\neg B \to \neg A)$.

10) $A \to (\neg B \to \neg(A \to B))$.

11) $(A \to B) \to ((\neg B \to A) \to B)$.

12) $((A \to B) \to A) \to A$.

Глава 4. МЕТОД РЕЗОЛЮЦИЙ В ЛОГИКЕ ВЫСКАЗЫВАНИЙ

Метод резолюций – это метод автоматического доказательства теорем – основы логического программирования. Это алгоритм, проверяющий отношение выводимости $\Gamma \vdash A$. В общем случае алгоритм автоматического доказательства теорем не существует, но для формальных теорий с несложной структурой (таких как исчисление высказываний, исчисление предикатов с одним одноместным предикатом) подобные алгоритмы известны.

Вообще говоря, в построенном в главе 3 исчислении высказываний (благодаря полноте исчисления) проверка выводимости формулы состоит в проверке того, является ли формула тавтологией или нет. Это можно легко установить по таблицам истинности. Но этот метод не обеспечивает построения вывода формулы.

Метод резолюций – классический алгоритм автоматического доказательства теорем. Для простоты изложения рассмотрим его для исчисления высказываний. Для любого множества формул Γ и любой формулы A метод дает утвердительный ответ, если $\Gamma \vdash A$, и дает отрицательный ответ, если неверно, что $\Gamma \vdash A$.

Теорема о доказательстве от противного. Если $\Gamma, \neg A \vdash F$, где F – тождественно ложная формула, то $\Gamma \vdash A$.

Доказательство. Доказательство проведем для частного случая, когда Γ представляет собой одну формулу. По теореме дедукции, $\Gamma, \neg A \vdash F \Leftrightarrow \Gamma \to (\neg A \to F)$ – тавтология. Преобразуем правую часть равносильности, учитывая, что формула F тождественно ложна.

$\Gamma \to (\neg A \to F) = \neg\Gamma \vee (\neg\neg A \vee F) = \neg\Gamma \vee A \vee F = \neg\Gamma \vee A =$

$= \Gamma \to A$ – тавтология $\Leftrightarrow \Gamma \vdash A$, что и требовалось доказать.

Как правило, в качестве формулы F используют пустую формулу , которая не имеет никакого значения ни в какой интерпретации, и, по определению, является противоречием.

Метод резолюций использует специальную форму формул, которая называется предложением.

Определение. Предложением называется дизъюнкция формул вида A или $\neg A$, где A – атом (буква).

Любая формула исчисления высказываний может быть преобразована в предложение следующей последовательностью действий:

1. Замена импликации по формуле: $A \to B = \neg A \vee B$ (проверьте самостоятельно). В результате в формуле остаются связки: $\neg$, $\vee$, $\wedge$.
2. Преобразование выражений с инверсиями по закону двойного отрицания: $\neg\neg A = A$, законам де Моргана: $\neg(A \vee B) = \neg A \wedge \neg B$, $\neg(A \wedge B) = \neg A \vee \neg B$.

   В результате инверсии остаются только перед буквами.
3. Приведение формулы к конъюнктивной нормальной форме с помощью дистрибутивных законов:

   $A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C)$,

   $A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$.
4. Преобразование конъюнктивной нормальной формы во множество предложений: $AB \Rightarrow A, B$.

Напомню, что связки $\vee$, $\wedge$ используются здесь для удобства записи.

Определение. Множество формул называется невыполнимым, если оно не имеет модели, то есть интерпретации, в которой все формулы истинны.

Без доказательства приведем следующую теорему.

Теорема. Если из формулы A получено множество Δ предложений, то формула A тождественно ложна тогда и только тогда, когда множество Δ невыполнимо.

До сих пор мы пользовались только одним правилом вывода – Modus ponens. В других исчислениях высказываний имеют место и другие правила вывода.

Правило резолюций. Даны предложения: $C_1 = P \vee C_1'$, $C_2 = \neg P \vee C_2'$, где P – пропозициональная буква, C_1' и C_2' – предложения (в частности, пустые или содержащие только одну букву или ее отрицание). Правило резолюций формулируется так: C_1, $C_2 \vdash C_1' \vee C_2'$.

C_1, C_2 называются *резольвируемыми предложениями*, а $C_1' \vee C_2'$ – *резольвентой*.

Правило резолюций будем обозначать R.

Теорема. Резольвента логически следует из резольвируемых предложений.

Доказательство. В вышеприведенных обозначениях, нам нужно доказать, что $C_1 \to \left(C_2 \to \left(C_1' \vee C_2'\right)\right)$ – тавтология (по теореме дедукции).

Предположим, что $\left|C_1 \to \left(C_2 \to \left(C_1' \vee C_2'\right)\right)\right| = 0 \Leftrightarrow$

$$\Leftrightarrow \begin{cases} |C_1| = 1, \\ \left|C_2 \to \left(C_1' \vee C_2'\right)\right| = 0, \end{cases} \Leftrightarrow \begin{cases} |C_1| = 1, \\ |C_2| = 1, \\ \left|C_1' \vee C_2'\right| = 0, \end{cases} \Leftrightarrow \begin{cases} |C_1| = 1, \\ |C_2| = 1, \\ |C_1'| = 0, \\ |C_2'| = 0, \end{cases} \Leftrightarrow \begin{cases} |C_1' \vee P| = 1, \\ |C_2' \vee \neg P| = 1, \\ |C_1'| = 0, \\ |C_2'| = 0, \end{cases} \Leftrightarrow$$

$$\begin{cases} |P| = 1, \\ |\neg P| = 1, \\ |C_1'| = 0, \\ |C_2'| = 0, \end{cases} \Leftrightarrow \begin{cases} |P| = 1, \\ |P| = 0, \\ |C_1'| = 0, \\ |C_2'| = 0. \end{cases}$$

Полученное противоречие доказывает утверждение теоремы.

Правило резолюций применяется в опровержении методом резолюций – алгоритме, устанавливающем выводимость $\Gamma \vdash A$.

Запишем $\neg A$. Каждая формула из множества Γ и формула $\neg A$ независимо преобразуются во множество предложений. В этом множестве нужно найти резольвируемые предложения и применить к ним правило резолюций. Резольвенты добавляются во множество предложений до тех пор, пока не будет получено пустое предложение. Возможны два случая:

- Среди множества предложений нет резольвируемых. Вывод: теорема опровергнута, и формула A не выводима из множества формул Γ.
- Получено пустое предложение. Теорема доказана. Имеет место выводимость $\Gamma \vdash A$.

Примеры. 1. Методом резолюций доказать теорему $\vdash \neg A \to (A \to B)$.

Доказательство. Запишем инверсию исходной формулы:

$\neg\left(\neg A \to \left(A \to B\right)\right)$.

Заменим все импликации по соответствующей формуле:

$\neg\left(\neg A \to \left(A \to B\right)\right) = \neg\left(\neg\neg A \vee \left(\neg A \vee B\right)\right)$.

Применим закон двойного отрицания и закон де Моргана:

$$\neg(\neg A \to (A \to B)) = \neg(A \vee (\neg A \vee B)) = \neg A \wedge \neg(\neg A \vee B) =$$

$$= \neg A \wedge \neg\neg A \wedge \neg B = \neg A \wedge A \wedge \neg B.$$

Получаем предложения: $\neg A,\ A,\ \neg B$. Резольвируем их:

1. $\neg A$ – предложение.
2. A – предложение.
3. $\neg B$ – предложение.
4. . R 1, 2.

2. Методом резолюций доказать теорему

$$\vdash A \to (B \to A \wedge B).$$

Доказательство. Запишем инверсию исходной формулы:

$$\neg(A \to (B \to A \wedge B)).$$

Заменим все импликации по соответствующей формуле:

$$\neg(A \to (B \to A \wedge B)) = \neg(\neg A \vee (\neg B \vee A \wedge B)).$$

Применим закон двойного отрицания и закон де Моргана:

$$\neg(A \to (B \to A \wedge B)) = \neg\neg A \wedge \neg(\neg B \vee (A \wedge B)) =$$

$$= A \wedge B \wedge \neg(A \wedge B) = A \wedge B \wedge (\neg A \vee \neg B).$$

Получаем предложения: $A,\ B,\ \neg A \vee \neg B$.

1. A – предложение.
2. B – предложение.
3. $\neg A \vee \neg B$ – предложение.
4. $\neg B$. R 1, 3.
5. . R 2, 4.

Задачи

Методом резолюций доказать теоремы:

1) $\vdash (A \to B) \to ((B \to C) \to (A \to C))$.
2) $\vdash (A \to (B \to C)) \to (B \to (A \to C))$.
3) $\vdash (A \to B) \to (\neg B \to \neg A)$.
4) $\vdash (A \to B) \to ((C \to A) \to (C \to B))$.
5) $\vdash A \to (\neg A \to B)$.
6) $\vdash (A \to B) \to ((\neg A \to B) \to B)$.

7) $\vdash (A \to B) \to ((A \to (B \to C)) \to (A \to C))$.

8) $\vdash (A \to B) \wedge (A \to \neg B) \to \neg A$.

9) $\vdash (A \to (B \to C)) \to (A \wedge B \to C)$.

10) $\vdash (A \to B) \to ((B \to C) \to ((C \to D) \to (A \to D)))$.

11) $\vdash (A \to B) \to ((B \to C) \to (A \to C))$.

12) $\vdash (A \wedge B \to C) \to (A \to (B \to C))$.

13) $\vdash (A \to (B \to A)) \to A$.

14) $\vdash \neg A \vee A$.

15) $\vdash (A \to B) \vee (B \to A)$.

Глава 5. ПРЕДИКАТЫ

В исчислении высказываний нет предметных переменных, то есть переменных, которые могут принимать нелогические значения, например, числовые. Для того чтобы в логические исчисления могли быть включены нелогические константы и переменные, вводится понятие предиката.

Определение. *n-местным предикатом* на множестве X называется n-местная функция из множества X^n во множество $\{0,1\}$.

Примеры. 1. Предикат $A(x) = "x \le 2"$ на множестве $X = R$ – одноместный.

2. Предикат $B(x, y) = "xy > 0"$ на множестве $X = R^2$ – двуместный.

Если $X = \{0,1\}$, то n-местный предикат представляет собой n-местную булеву функцию.

Нульместный предикат представляет собой высказывание.

Для каждого предиката A областью истинности называется множество $Y = \{x \in X \,|\, A(x) = 1\}$, на котором предикат принимает значение 1.

Примеры. 1. Для предиката $A(x) = "x \le 2"$ на множестве $X = R$ область истинности $Y = \{x \in R \,|\, x \le 2\}$.

2. Для предиката $B(x, y) = "xy > 0"$ на множестве $X = R^2$ область истинности $Y = \{(x, y) \in R^2 \,|\, xy > 0\}$.

Поскольку множество значений любого предиката лежит во множестве $\{0,1\}$, то с предикатами можно производить все операции алгебры логики, и все известные свойства логиче-

ских операций обобщаются для предикатов. Рассмотрим эти свойства (для удобства в свойствах записываются одноместные предикаты):

3. Коммутативность:

$$P(x)\vee Q(x)=Q(x)\vee P(x),\ P(x)\wedge Q(x)=Q(x)\wedge P(x).$$

2. Ассоциативность:

$$P(x)\vee(Q(x)\vee R(x))=(P(x)\vee Q(x))\vee R(x),$$

$$P(x)\wedge(Q(x)\wedge R(x))=(P(x)\wedge Q(x))\wedge R(x).$$

3. Дистрибутивность:

$$P(x)\vee(Q(x)\wedge R(x))=(P(x)\vee Q(x))\wedge(P(x)\vee R(x)),$$

$$P(x)\wedge(Q(x)\vee R(x))=(P(x)\wedge Q(x))\vee(P(x)\wedge R(x)).$$

4. Идемпотентность: $P(x)\vee P(x)=P(x),\ P(x)\wedge P(x)=P(x).$

5. Закон двойного отрицания: $\neg\neg P(x)=P(x).$

6. Закон исключения третьего: $P(x)\vee\neg P(x)=1.$

7. Закон противоречия: $P(x)\wedge\neg P(x)=0.$

8. Законы де Моргана:

$$\neg(P(x)\vee Q(x))=\neg P(x)\wedge\neg Q(x),$$

$$\neg(P(x)\wedge Q(x))=\neg P(x)\vee\neg Q(x).$$

9. Свойства операций с логическими константами:

$$P(x)\vee 1=1,\ P(x)\vee 0=P(x),\ P(x)\wedge 1=P(x),\ P(x)\wedge 0=0.$$

Здесь $P(x)$, $Q(x)$ и $R(x)$ – любые предикаты.

В то же время, для предикатов определены операции специального вида, которые называются *кванторами*.

Пусть дан n-местный предикат $A(x_1,x_2,...,x_n)$ на множестве X, означающий, что для набора $(x_1,x_2,...,x_n)$ выполнено свойство A, и пусть x_i – одна из переменных. Тогда запись $\forall x_i A(x_1,x_2,...,x_n)$ означает, что для всех значений переменной x_i свойство A выполнено. Символ $\forall$ называется *квантором всеобщности* (*общности*). Предикат $\forall x_i A(x_1,x_2,...,x_n)$ является $(n-1)$-местным. Он зависит от переменных $x_1,x_2,...,x_{i-1},x_{i+1},...,x_n$. Если дан одноместный предикат $P(x)$, то утверждение $\forall xP(x)$ представляет собой нульместный предикат, то есть истинное или ложное высказывание.

Пример. На множестве $X = R$ дан предикат $A(x) = "x \le 2"$. Высказывание $\forall x(x \le 2)$ ложно.

Пусть дан n-местный предикат $A(x_1, x_2, ..., x_n)$ на множестве X, означающий, что для набора $(x_1, x_2, ..., x_n)$ выполнено свойство A, и пусть x_i – одна из переменных. Тогда запись $\exists x_i A(x_1, x_2, ..., x_n)$ означает, что существует значение переменной x_i, такое, что выполняется свойство A. Символ $\exists$ называется *квантором существования*. Предикат $\exists x_i A(x_1, x_2, ..., x_n)$ является $(n-1)$-местным. Если дан одноместный предикат $P(x)$, то утверждение $\exists x P(x)$ представляет собой нульместный предикат, то есть истинное или ложное высказывание.

Пример. На множестве $X = R$ дан предикат $A(x) = "x \le 2"$. Высказывание $\exists x(x \le 2)$ истинно.

Отметим, что запись $\forall x A$ ($\exists x A$) не подразумевает, что в формуле A есть переменная x.

Пусть дана запись $\forall x A$ (или $\exists x A$). Переменная x называется *переменной в кванторе*, а A – *областью действия квантора.*

Имеют место эквивалентности:

$\exists x_i A = \neg \forall x_i \neg A.$ $\qquad$ $\forall x_i A = \neg \exists x_i \neg A.$

$\neg \exists x_i A = \forall x_i \neg A.$ $\qquad$ $\neg \forall x_i A = \exists x_i \neg A.$

Отметим, что список переменных в предикате A мы будем указывать не всегда.

Предикат называется *тождественно истинным* (*тождественно ложным*), если при всех возможных значениях переменных он принимает значение 1(0).

Теорема. Пусть $A(x_1, x_2, ..., x_n)$ – n-местный предикат, x_i – переменная в предикате. Тогда предикат $\forall x_i A(x_1, x_2, ..., x_n) \to A(x_1, x_2, ..., x_n)$ является тождественно истинным.

Доказательство. Возьмем произвольный набор значений $(x_1^0, x_2^0, ..., x_i^0, ..., x_n^0)$ переменных $(x_1, x_2, ..., x_i, ..., x_n)$. Подставим этот набор в предикат $\forall x_i A(x_1, x_2, ..., x_n) \to A(x_1, x_2, ..., x_n)$. Получим высказывание:

$\forall x_i A(x_1^0, x_2^0, ..., x_i, ..., x_n^0) \to A(x_1^0, x_2^0, ..., x_i^0, ..., x_n^0) = B_0.$

Покажем, что это высказывание истинно. Возможны два случая.

1. $\forall x_i A(x_1^0, x_2^0, ..., x_i, ..., x_n^0) = 0$, следовательно $B_0 = 1$.
2. $\forall x_i A(x_1^0, x_2^0, ..., x_i, ..., x_n^0) = 1$.

Соотношение выполнено при любых значениях x_i, следовательно, и при значении $x_i = x_i^0$. При подстановке этого значения получаем:

$A(x_1^0, x_2^0, ..., x_i^0, ..., x_n^0) = 1$.

Следовательно, по свойству импликации получаем, что $B_0 = 1$, что и требовалось доказать.

Теорема. Пусть $A(x_1, x_2, ..., x_n)$ – n-местный предикат, x_i – переменная в предикате. Тогда предикат $A(x_1, x_2, ..., x_n) \to \exists x_i A(x_1, x_2, ..., x_n)$ является тождественно истинным.

Доказательство аналогично доказательству предыдущей теоремы.

Предикат называется *выполнимым*, если при некоторых значениях переменных он принимает значение 1.

Пример. Найти значение высказывания $\exists x \forall y \exists z \left(xz = y^3 \right)$. Предикат $xz = y^3$ определен на множестве $X = N$.

Решение. Пусть $\exists x \forall y \exists z \left(xz = y^3 \right) = 1$. Эквивалентность имеет место тогда и только тогда, когда для некоторого x_0 $\forall y \exists z \left(x_0 z = y^3 \right) = 1$. Это означает, что для некоторого x_0 предикат $\exists z \left(x_0 z = y^3 \right)$ является тождественно истинным относительно y, то есть для некоторого x_0 и для произвольного y_0 $\exists z \left(x_0 z = y_0{}^3 \right) = 1$. Последнее равносильно тому, что предикат $\left(x_0 z = y_0{}^3 \right)$ выполним. Предикат действительно является выполнимым, поскольку он определен на множестве натуральных чисел. Таким образом, поскольку все переходы были равносильными, исходное высказывание истинно.

Предикаты могут быть выражены с помощью так называемых предикатных формул. Строгое определение формулы исчисления предикатов будет дано в следующей главе. Пока нужно учитывать, что формула становится предикатом, когда все переменные определены на некотором множестве, и определены все предикаты, входящие в формулу.

Справедливы эквивалентности:

$\forall x \forall y P(x, y) = \forall y \forall x P(x, y)$, $\exists x \exists y P(x, y) = \exists y \exists x P(x, y)$.

Разноименные кванторы можно переставлять только следующим образом:

$\exists x \forall y P(x, y) \to \forall y \exists x P(x, y)$, $\exists y \forall x P(x, y) \to \forall x \exists y P(x, y)$.

Обратные формулы неверны.

Пример. Очевидно, что высказывание $\forall x\exists y(x+y=0)$ ($X=R$) истинно. Поменяем кванторы местами. Получим высказывание $\exists y\forall x(x+y=0)$, которое является ложным.

Выражения с кванторами можно преобразовывать следующим образом:

$\forall x(P(x)\wedge Q(x))=\forall xP(x)\wedge\forall xQ(x)$, $\exists x(P(x)\vee Q(x))=\exists xP(x)\vee\exists xQ(x)$.

Докажем первую эквивалентность. Пусть предикаты $P(x)$ и $Q(x)$ одновременно тождественно истинны. Тогда тождественно истинным будет и предикат $P(x)\wedge Q(x)$, следовательно, истинными будут высказывания $\forall x(P(x)\wedge Q(x))$, $\forall xP(x)$, $\forall xQ(x)$, а также $\forall xP(x)\wedge\forall xQ(x)$.

Пусть теперь хотя бы один из предикатов (например, $P(x)$) не является тождественно истинным. Тогда (по свойствам конъюнкции) тождественно истинным не будет и предикат $P(x)\wedge Q(x)$, следовательно, ложным будет высказывание $\forall x(P(x)\wedge Q(x))$. Высказывания $\forall xP(x)$ и $\forall xP(x)\wedge\forall xQ(x)$ также будут ложными.

Таким образом, обе части эквивалентности одновременно истинны или ложны, и эквивалентность доказана.

Замечание. Формула $\forall x(P(x)\vee Q(x))$ не эквивалентна формуле $\forall xP(x)\vee\forall xQ(x)$.

Доказательство. Рассмотрим обе формулы на множестве R. Пусть предикат $P(x)=''x<0''$, а предикат $Q(x)=''x\geq 0''$. Оба предиката не являются тождественно истинными. Предикат $P(x)\vee Q(x)=''x\in R''$ – тождественно истинный, и высказывание $\forall x(P(x)\vee Q(x))$ истинно. Высказывания $\forall xP(x)$ и $\forall xQ(x)$ ложны, следовательно, ложно и высказывание $\forall xP(x)\vee\forall xQ(x)$. Таким образом, построен пример, когда формулы $\forall x(P(x)\vee Q(x))$ и $\forall xP(x)\vee\forall xQ(x)$ принимают различные значения.

Тем не менее, справедливы эквивалентности:

$$\forall xP(x)\vee\forall xQ(x)=\forall xP(x)\vee\forall yQ(y)=\forall x(P(x)\vee\forall yQ(y))=$$
$$=\forall x\forall y(P(x)\vee Q(y)).$$

Аналогично, формулы $\exists x(P(x)\wedge Q(x))$ и $\exists xP(x)\wedge\exists xQ(x)$ не эквивалентны. Но справедливы эквивалентности:

$$\exists xP(x)\wedge\exists xQ(x)=\exists xP(x)\wedge\exists yQ(y)=\exists x(P(x)\wedge\exists yQ(y))=$$
$$=\exists x\exists y(P(x)\wedge Q(y)).$$

Имеют место формулы:

$\forall x\big(P(x)\wedge C\big)=\forall xP(x)\wedge C$, $\exists x\big(P(x)\vee C\big)=\exists xP(x)\vee C$,

$\forall x\big(P(x)\vee C\big)=\forall xP(x)\vee C$, $\exists x\big(P(x)\wedge C\big)=\exists xP(x)\wedge C$.

Здесь C не содержит переменной x.

Определение. Предикатная формула находится в *приведенной форме*, если в ней использованы только кванторные операции, а также операции инверсии, конъюнкции, дизъюнкции, причем инверсия относится только к предикатным буквам.

Определение. Предикатная формула находится в *предваренной форме* (*предваренной нормальной форме*), если она имеет вид $Q_1x_1Q_2x_2...Q_kx_kA$, где $Q_1,Q_2,...,Q_k$ – кванторы всеобщности или существования, а формула A находится в приведенной форме и не содержит кванторов.

Пример. Записать формулу

$$A=\exists x\forall yP(x,y)\to\forall x\exists yQ(x,y)\vee R(x)$$

в предваренной нормальной форме.

Решение. $A=\exists x\forall yP(x,y)\to\forall x\exists yQ(x,y)\vee R(x)=$

$=\neg\exists x\forall yP(x,y)\vee\forall x\exists yQ(x,y)\vee R(x)=$

$=\forall x\neg\forall yP(x,y)\vee\forall x\exists yQ(x,y)\vee R(x)=$

$=\forall x\exists y\neg P(x,y)\vee\forall x\exists yQ(x,y)\vee R(x).$

Полученная формула записана в приведенной форме. Для того чтобы квантор всеобщности можно было вынести за скобки, переобозначим переменные и выполним преобразования:

$A=\forall t\exists y\neg P(t,y)\vee\forall z\exists yQ(z,y)\vee R(x)=$

$=\forall t\big(\exists y\neg P(t,y)\vee\forall z\exists yQ(z,y)\vee R(x)\big)=$

$=\forall t\forall z\big(\exists y\neg P(t,y)\vee\exists yQ(z,y)\vee R(x)\big)=$

$=\forall t\forall z\exists y\big(\neg P(t,y)\vee Q(z,y)\vee R(x)\big).$

Рассмотрим предикат $P(x)$, определенный на конечном множестве $X=\{a_1,a_2,...,a_n\}$. Если предикат $P(x)$ является тождественно истинным, то истинными будут высказывания $P(a_1)$, $P(a_2)$, ..., $P(a_n)$. При этом истинными будут высказывания $\forall xP(x)$ и конъюнкция $P(a_1)\ P(a_2)\dots\ P(a_n)$.

Если же хотя бы для одного элемента $a_k\in M$ $P(a_k)$ будет ложно, то ложными будут высказывания $\forall xP(x)$ и $P(a_1)\ P(a_2)\dots\ P(a_n)$.

Таким образом, имеет место эквивалентность $\forall xP(x)=P(a_1)\ P(a_2)\dots\ P(a_n)$.

Справедлива и аналогичная эквивалентность

$$\exists x P(x) = P(a_1) \vee P(a_2) \vee \ldots \vee P(a_n).$$

Пример. Найти предикат, логически эквивалентный предикату $\exists z \forall y A(y,z) \vee \forall x B(x,y)$, но не содержащий кванторов. Предикаты A и B определены на множестве $\{a,b,c\}$.

Решение. $\exists z \forall y A(y,z) \vee \forall x B(x,y) =$

$$= \big(\forall y A(y,a) \vee \forall y A(y,b) \vee \forall y A(y,c)\big) \vee B(a,y)B(b,y)B(c,y) =$$
$$= A(a,a)A(b,a)A(c,a) \vee A(a,b)A(b,b)A(c,b) \vee A(a,c)A(b,c)A(c,c) \vee$$
$$\vee B(a,y)B(b,y)B(c,y).$$

С помощью предикатов можно записывать различные математические утверждения.

Пример. Покажем, как можно записать утверждение: «числовая последовательность $\{x_n\}$ имеет пределом число a ($\lim\limits_{n\to\infty} x_n = a$)».

Решение. Запишем данное утверждение с помощью кванторов и обозначим его A:

$$A = \forall \varepsilon \exists N \forall n \big(\varepsilon > 0 \to \big(n > N \to |x_n - a| < \varepsilon\big)\big).$$

Запишем инверсию данного высказывания:

$$\neg A = \neg \forall \varepsilon \exists N \forall n \big(\varepsilon > 0 \to \big(n > N \to |x_n - a| < \varepsilon\big)\big) =$$
$$= \exists \varepsilon \neg \exists N \forall n \big(\varepsilon > 0 \to \big(n > N \to |x_n - a| < \varepsilon\big)\big) =$$
$$= \exists \varepsilon \forall N \neg \forall n \big(\varepsilon > 0 \to \big(n > N \to |x_n - a| < \varepsilon\big)\big) =$$
$$= \exists \varepsilon \forall N \exists n \neg \big(\varepsilon > 0 \to \big(n > N \to |x_n - a| < \varepsilon\big)\big).$$

По известным формулам, инверсия импликации преобразуется следующим образом:

$$\neg(K \to M) = \neg(\neg K \vee M) = \neg\neg K \wedge \neg M = K \wedge \neg M.$$

Отсюда получаем:

$$\neg A = \exists \varepsilon \forall N \exists n \big((\varepsilon > 0) \wedge \neg\big(n > N \to |x_n - a| < \varepsilon\big)\big) =$$
$$= \exists \varepsilon \forall N \exists n \big((\varepsilon > 0) \wedge (n > N) \wedge \neg\big(|x_n - a| < \varepsilon\big)\big) =$$
$$= \exists \varepsilon \forall N \exists n \big((\varepsilon > 0) \wedge (n > N) \wedge \big(|x_n - a| \geq \varepsilon\big)\big).$$

Утверждение $\neg A$ означает, что $\lim\limits_{n\to\infty} x_n \neq a$, то есть число a не является пределом числовой последовательности $\{x_n\}$.

Задачи

1. Среди следующих предложений выделить предикаты, и для каждого предиката установить местность и область истинности, если $X = R$. Для двуместных предикатов изобразить область истинности графически.

1) $x + 2 = 0$.

2) При $x = 0$ выполняется равенство $x - 2 = 0$.

3) $x^3 - 8 = 0$.

4) $\exists x\left(x^3 - 8 = 0\right)$.

5) $x - y = 1$.

6) $x^2 - 5x + 7$.

7) $\sin^2 x + \cos^2 x = 1$.

8) $x - y^2 \geq 0$.

9) Однозначное число x является простым.

10) $\dfrac{x^2 - 1}{x^2 - 2x + 1} = 0$.

2. Определить значение высказывания, полученного из трехместного предиката на множестве X.

1) $\exists x \forall y \exists z\left(xz = y\right),\ X = N$.

2) $\forall z \exists x \forall y\left(xz = y^2\right),\ X = Z$.

3) $\forall z \exists y \forall x\left(xy > xz\right),\ X = N$.

4) $\exists x \forall y \forall z\left(xyz < yz\right),\ X = Z$.

5) $\forall x \forall z \exists y\left(x + y = z\right),\ X = N$.

6) $\exists x \exists y \exists z\left(xyz = yz\right),\ X = R$.

7) $\exists z \forall x \exists y\left(x + y > z + y\right),\ X = R$.

8) $\forall y \exists z \forall x\left(x^2 + y^2 = z\right),\ X = R$.

9) $\forall x \forall y \forall z\left(x(y + z) = xy + xz\right),\ X = R$.

10) $\forall x \forall z \exists y\left(x + y = z\right), X = R$.

3. Записать инверсию формулы в предваренной нормальной форме.

1) $\exists x \neg A(x)$.

2) $\exists x\big(A(x) \neg B(x)\big)$.

3) $\forall x \neg A(x)$.

4) $\forall x\big(A(x) \to \forall y B(y)\big)$.

5) $\exists x\big(A(x) B(x) C(x)\big)$.

6) $\forall x\big(A(x) \to B(x)\big)$.

7) $\exists x\big(A(x) \leftrightarrow B(x)\big)$.

8) $\forall x\big(A(x) \vee \exists y B(y)\big)$.

9) $\forall x\big(A(x) \to B(x)\big) \wedge \exists x\big(C(x) \wedge \neg D(x)\big)$.

10) $\forall x \exists y \forall z\big(A(x, y, z) \to B(x, y, z)\big)$.

4. Записать формулу в приведенной форме, если это необходимо, а затем преобразовать к предваренной форме.

1) $\exists x \exists z R(x, y, z) \to \forall x P(x, y)$.

2) $\forall x \exists y R(x, y, z) \to \exists x P(x, y)$.

3) $\forall x P(x, y) \wedge \exists y Q(x, y)$.

4) $\exists x \forall y P(x, y) \wedge \exists x Q(x, y)$.

5) $\forall y R(x, y) \to \exists x \forall z P(x, y, z)$.

6) $\forall x \exists y P(x, y) \wedge \exists x \exists y Q(x, y)$.

7) $\forall x P(x, y) \to \exists x \forall z Q(x, z)$.

8) $\exists x \forall y P(x, y) \vee \neg \forall x \exists y Q(x, y)$.

9) $\exists x \forall z P(x, y, z) \to \big(\forall z R(x, y, z) \vee Q(x, y)\big)$.

10) $\forall x P(x, y, z) \to \exists x \forall z Q(x, y, z)$.

5. Найти предикат, не содержащий кванторов, логически эквивалентный данному предикату. Предикаты A и B определены на множестве $\{a, b, c\}$.

1) $\forall y A(x, y) \to \forall x \exists z B(x, z)$.

2) $\forall x \exists y A(x, y) \wedge \forall z B(x, z)$.

3) $\forall x A(x, y) \vee \forall x \exists z B(x, z)$.

4) $\forall y \exists x A(x, y) \to \exists x \forall z B(x, z)$.

5) $\exists y \forall x A(x, y) \vee \forall z B(x, z)$.

6) $\forall x A(x, y) \vee \forall x \forall z B(x, z)$.

7) $\exists x A(x, y) \wedge \exists z \forall x B(x, z)$.

8) $\forall x A(x, z) \to \exists x \exists y B(x, y)$.

9) $\forall x \exists y A(x, y) \wedge \forall z B(x, z)$.

10) $\forall y A(x, y) \vee \exists x \forall z B(x, z)$.

6. Записать с помощью кванторов следующие утверждения и их отрицания.

1) Функция $f(x)$ возрастает на интервале (a,b).

2) Функция $f(x)$ непрерывна на интервале (a,b).

3) Множество A является собственным подмножеством множества B.

4) Точка x_0 является точкой экстремума функции $f(x)$.

5) Функция $f(x)$ достигает наибольшего значения на отрезке $[a,b]$ в точке x_0.

6) Функция $f(x)$ дифференцируема в точке x_0.

7) Бинарное отношение ρ является симметричным.

8) Функция $f(x)$ ограничена на множестве R.

9) Булева функция $f(x_1, x_2, ..., x_k)$ самодвойственна.

10) Множества A и B не пересекаются.

7. Доказать эквивалентность

$\exists x\big(P(x) \vee Q(x)\big) = \exists x P(x) \vee \exists x Q(x)$.

8. Доказать, что не эквивалентны формулы $\exists x\big(P(x) \wedge Q(x)\big)$ и $\exists x P(x) \wedge \exists x Q(x)$.

Глава 6. ИСЧИСЛЕНИЕ ПРЕДИКАТОВ

Рассмотрим построение теории первого порядка.

Компонентами теории первого порядка являются следующие.

1. Алфавит составляют:

- Предметные константы – буквы начала латинского алфавита с натуральными индексами: a_1, a_2, …, b_1, b_2, … Предметные символы – это имена (обозначения) предметов.
- Предметные переменные – буквы конца латинского алфавита с натуральными индексами: x_1, x_2, …, y_1, y_2, …

- Функциональные буквы – строчные буквы латинского алфавита с натуральными индексами (верхний индекс указывает число переменных, нижний – номер функциональной буквы): $f_k^{(n)}$, $g_k^{(n)}$, …
- Предикатные буквы – заглавные буквы латинского алфавита с натуральными индексами (верхний индекс указывает число переменных, нижний – номер предикатной буквы): $A_k^{(n)}$, $B_k^{(n)}$, $C_k^{(n)}$,... (индексы можно не указывать).
- Логические связки: $\neg, \rightarrow$.
- Квантор всеобщности $\forall$.
- Синтаксические символы – скобки (,) и запятая.

2. Формула определяется несколькими этапами. Вначале вводится понятие терма.

Определение. 1) Предметные константы и предметные переменные есть *термы*.

2) Если t_1, t_2, …, t_n, – термы, $f_k^{(n)}$ – функциональная буква, то $f_k^{(n)}(t_1, t_2, ..., t_n)$ – терм.

3) Символ является термом тогда и только тогда, когда это следует из 1) и 2).

Примеры. 1. Пусть x_1 – предметная переменная, a_1 – предметная константа, $f_1^{(1)}$, $f_1^{(2)}$ – функциональные буквы. Тогда $f_1^{(1)}(x_1)$, $f_1^{(2)}\left(f_1^{(1)}(x_1), a_1\right)$ – термы.

2. Пусть x – предметная переменная, a – предметная константа, $\sin$, $\cos$ – функциональные буквы. Тогда $\sin x$, $\cos ax$ – термы. Здесь символы $\sin$, $\cos$ имеют только формальный смысл и не интерпретируются как обозначения тригонометрических функций.

Определение. Если t_1, t_2, …, t_n, – термы, $A_k^{(n)}$ – предикатная буква, то символ $A_k^{(n)}(t_1, t_2, ..., t_n)$ называется элементарной формулой.

Другими словами, элементарная формула образуется при применении предикатной буквы к термам.

Примеры. 1. В условиях первого примера, если $A_1^{(2)}$ – предикатная буква, то $A_1^{(2)}\left(f_1^{(1)}(x_1), f_2^{(2)}(x_1, x_2)\right)$ – элементарная формула.

2. В условиях второго примера, если $A_2^{(2)} = "\leq"$ – предикатная буква, то $\sin x \leq \cos ax$ – элементарная формула.

Теперь определим формулу логики предикатов.

Определение. 1) Всякая элементарная формула есть формула.

2) Если A, B – формулы, то формулами являются также символы $\neg A$, $A \to B$, $\forall yA$.

3) Символ является формулой тогда и только тогда, когда это следует из 1) и 2).

Примеры. 1. В условиях первого примера, $\forall x_1 A_1^{(2)}\left(f_1^{(1)}(x_1), f_2^{(2)}(x_1, x_2)\right)$ – формула.

2. В условиях второго примера, $\forall x(\sin x \le \cos ax)$ – формула.

В теории первого порядка, как и в исчислении высказываний, допускаются формулы с другими логическими связками, а также допускается использование квантора существования. Известна формула (см. **Глава 5. ПРЕДИКАТЫ**).

Здесь мы ненадолго отвлечемся от построения теории первого порядка и рассмотрим некоторые понятия, связанные с формулами.

Определение. Пусть A – формула, x_i – переменная, которая входит в формулу A (один или несколько раз). Вхождение x_i в формулу A называется *связанным*, если либо x_i – переменная в кванторе ($\forall x_i$), либо x_i находится в области действия квантора $\forall x_i$. Если вхождение x_i в A не связано, то оно называется свободным.

Пример. В формуле $A^{(2)}(x_i, x_j)$ вхождения обеих переменных свободные.

В формуле $\left(\forall x_i A^{(2)}(x_i, x_j)\right) \to A^{(1)}(x_i)$ вхождения переменной x_i в посылку связаны, а вхождение в следствие свободно. Вхождение переменной x_j свободно, так как отсутствует квантор $\forall x_j$.

В формуле $\exists x_j \forall x_i A^{(2)}(x_i, x_j)$ вхождения обеих переменных связаны.

Пусть A – формула, x_i – переменная в формуле A, t – терм. Введем обозначение $A(x_i)$. Тогда $A(t)$ – результат подстановки t вместо всех свободных вхождений x_i в формулу A.

Пример. Рассмотрим подстановку t вместо всех свободных вхождений x_i в формулы из предыдущего примера.

В формуле $A^{(2)}(x_i, x_j)$ вхождение x_i свободное, следовательно, получаем $A^{(2)}(t, x_j)$.

В формуле $\left(\forall x_i A^{(2)}\left(x_i, x_j\right)\right) \to A^{(1)}\left(x_i\right)$ вхождения переменной x_i в посылку связаны, а вхождение в следствие свободно. Получаем: $\left(\forall x_i A^{(2)}\left(x_i, x_j\right)\right) \to A^{(1)}(t)$.

В формуле $\exists x_j \forall x_i A^{(2)}\left(x_i, x_j\right)$ вхождения обеих переменных связаны, поэтому осуществить подстановку t невозможно.

Определение. Терм t называется свободным для переменной x_i в формуле A тогда и только тогда, когда никакое свободное вхождение x_i в формулу A не лежит в области действия квантора $\forall x_j$, где x_j – переменная в терме t.

Пример. Рассмотрим формулу $\forall x_1 A\left(x_1, x_2\right)$ и терм $t = f^{(2)}\left(x_1, x_2\right)$. t не свободен для переменной x_2 в данной формуле, так как x_2 лежит в области действия квантора, тем более t не свободен для переменной x_1.

Пусть теперь дан терм $t = x_3$. t свободен для переменной x_2.

Уточним понятие интерпретации для множества формул Γ теории первого порядка.

Определение. *Интерпретацией* множества формул Γ называется область интерпретации X и заданное на ней соответствие, которое каждой предикатной букве $A_k^{(n)}$ ставит в соответствие n-местный предикат на X, каждой функциональной букве $f_k^{(n)}$ – n-местную функцию на X, каждой предметной константе a_i – элемент множества X.

При интерпретации формулы превращаются в предикаты на множестве X. Если формула не имеет свободных переменных, то после интерпретации она превращается в высказывание.

Пример. На множестве $X = R$ рассмотрим формулу $\forall x A(x, y)$. Интерпретируем эту формулу следующим образом: $A =$"$\le$". Тогда мы получим предикат $\forall x(x \le y)$.

Рассмотрим теперь формулу $\forall x \exists y A(x, y)$. При интерпретации она превращается в истинное высказывание $\forall x \exists y(x \le y)$.

Определение. Интерпретация называется *моделью* формальной теории (или некоторого множества формул), если все формулы формальной теории (или множества формул) истинны в данной интерпретации.

Определение. Формула называется *общезначимой* (*логически общезначимой*), если она истинна в любой интерпретации.

Определение. Формулы A и B называются *логически эквивалентными* тогда и только тогда, когда формула $A \leftrightarrow B$ логически общезначима.

Справедлива теорема, аналогичная теореме из логики высказываний.

Теорема. Отношение логической эквивалентности является отношением эквивалентности.

Определение. Говорят, что формула A *логически влечет* формулу B (из формулы A *логически следует* формула B), если формула $A \to B$ является логически общезначимой.

Теорема. Отношение логического следствия является отношением предпорядка.

Определение. Формула называется *противоречивой*, если она ложна в любой интерпретации.

Теорема. Пусть A – формула, x_i – переменная в формуле A, терм t свободен для переменной x_i в формуле A. Тогда формула $\forall x_i A(x_i) \to A(t)$ общезначима.

Доказательство. Пусть имеется некоторая интерпретация исходной формулы, то есть множество X и $A(x_i)$ – предикат на X. Покажем, что $\forall x_i A(x_i) \to A(t)$ – тождественно истинный предикат. Возьмем произвольный набор значений $(x_1^0, x_2^0, ..., x_i^0, ..., x_n^0)$ переменных $x_1, x_2, ..., x_i, ..., x_n$. Подставим этот набор в предикат. Получим высказывание:

$$\forall x_i A(x_1^0, x_2^0, ..., x_i, ..., x_n^0) \to A(t)(x_1^0, x_2^0, ..., x_i^0, ..., x_n^0) = B_0.$$

Покажем, что это высказывание истинно. Возможны два случая.

1. $\forall x_i A(x_1^0, x_2^0, ..., x_i, ..., x_n^0) = 0$, следовательно $B_0 = 1$.
2. $\forall x_i A(x_1^0, x_2^0, ..., x_i, ..., x_n^0) = 1$.

Соотношение выполнено при любых значениях x_i. Подставим этот набор значений в терм t: $t(x_1^0, x_2^0, ..., x_i, ..., x_n^0)$. Подставим последнее выражение в предикат $A(x_1^0, x_2^0, ..., x_i, ..., x_n^0)$. Получим:

$$A(x_1^0, x_2^0, ..., t(x_1^0, x_2^0, ..., x_i, ..., x_n^0), ..., x_n^0) = 1.$$

Но, поскольку терм t свободен для переменной x_i в формуле A, получаем:

$$A(x_1^0, x_2^0, ..., t(x_1^0, x_2^0, ..., x_i, ..., x_n^0), ..., x_n^0) = A(t)(x_1^0, x_2^0, ..., x_n^0) = 1.$$

Следовательно, по свойству импликации получаем, что $B_0 = 1$, что и требовалось доказать.

Теорема. Пусть x_i не является свободной переменной в формуле A, B – некоторая формула. Тогда формула $\forall x_i (A \to B) \to (A \to \forall x_i B)$ общезначима.

Доказательство аналогично доказательству предыдущей теоремы.

Теперь мы можем вернуться к построению теории первого порядка.

3. Аксиомы теории первого порядка делятся на два класса:

- Логические аксиомы:

1) $A \to (B \to A)$.

2) $(A \to (B \to C)) \to ((A \to B) \to (A \to C))$.

3) $(\neg B \to \neg A) \to ((\neg B \to A) \to B)$.

4) $\forall x_i A(x_i) \to A(t)$, где терм t свободен для переменной x_i в формуле $A(x_i)$.

5) $\forall x_i (A \to B) \to (A \to \forall x_i B)$, где x_i – несвободная переменная в формуле A.

Отметим, что аксиомы 1) – 3) – тавтологии, 4) и 5) – общезначимые формулы.

- Собственные аксиомы.

У каждой теории первого порядка свои собственные аксиомы.

4. Правила вывода.

1) Modus ponens (MP).

$A, A \to B \vdash B$.

2) Правило обобщения Gen.

$A \vdash \forall x A$.

Определение. Теория первого порядка без собственных аксиом называется исчислением предикатов первого порядка (или чистым исчислением предикатов).

Без доказательства приведем теоремы.

Теорема. Всякая теорема исчисления предикатов логически общезначима, то есть исчисление предикатов непротиворечиво.

Теорема о полноте. Всякая логически общезначимая формула является теоремой исчисления предикатов.

Рассмотрим несколько примеров теорий первого порядка с собственными аксиомами, (приведем только собственные аксиомы). Для удобства вместо предикатных и функциональных букв будем записывать привычные символы.

Теория равенства

Теория равенства – теория первого порядка с предикатной буквой $A_1^{(2)} = "="$.

Собственные аксиомы следующие:

1) $\forall x (x = x)$.

2) $x = y \Rightarrow A(x) = A(y)$.

Здесь A – произвольная предикатная буква.

Формальная арифметика

Формальная арифметика – теория первого порядка со следующими специальными символами.

1) Предметная константа 0.
2) Двуместные функциональные буквы $f_1^{(2)}(x,y)=x+y$ и $f_2^{(2)}(x,y)=x\times y$, одноместная функциональная буква $f_1^{(1)}(x)=x'$.
3) Двуместная предикатная буква $A_1^{(2)}=''='')$.

Собственные аксиомы следующие:

1) $\big(P(0)\wedge\forall x\big(P(x)\to P(x')\big)\big)\to\forall xP(x)$.
2) $\forall x_1\forall x_2\big(x_1'=x_2'\to x_1=x_2\big)$.
3) $\neg\big(x_1'=0\big)$.
4) $\forall x_1\forall x_2\forall x_3\big(x_1=x_2\to\big(x_2=x_3\to x_1=x_3\big)\big)$.
5) $\forall x_1\forall x_2\big(x_1=x_2\to x_1'=x_2'\big)$.
6) $\forall x_1\big(x_1+0=x_1\big)$.
7) $\forall x_1\forall x_2\Big(x_1+x_2'=\big(x_1+x_2\big)'\Big)$.
8) $\forall x_1\big(x_1\times 0=0\big)$.
9) $\forall x_1\forall x_2\big(x_1\times x_2'=x_1\times x_2+x_1\big)$.

Здесь P – произвольная предикатная буква.

Теория частично упорядоченных множеств

Теория частично упорядоченных множеств – это теория первого порядка с двумя предикатными буквами $A_1^{(2)}=''\le''$, $A_1^{(2)}=''='')$.

Собственные аксиомы следующие:

1) $\forall x_1\big(x_1=x_1\big)$.
2) $\forall x_1\forall x_2\big(x_1=x_2\to x_2=x_1\big)$.
3) $\forall x_1\forall x_2\forall x_3\big(x_1=x_2\to\big(x_2=x_3\to x_1=x_3\big)\big)$.
4) $\forall x_1\big(x_1\le x_1\big)$.
5) $\forall x_1\forall x_2\big(x_1\le x_2\to\big(x_2\le x_1\to x_1=x_2\big)\big)$.
6) $\forall x_1\forall x_2\forall x_3\big(x_1\le x_2\to\big(x_2\le x_3\to x_1\le x_3\big)\big)$.

Моделью данной теории является частично упорядоченное множество.

Для теорий первого порядка справедлива следующая теорема.

Теорема Геделя о неполноте. Во всякой достаточно богатой теории первого порядка (в частности, во всякой теории, включающей формальную арифметику), существует такая истинная формула A, что ни A, ни $\neg A$ не являются выводимыми в данной теории.

Задачи

1. Укажите, какие из следующих выражений являются формулами исчисления предикатов. В каждой формуле укажите свободные и связанные вхождения переменных:

1) $\forall x \forall y P(x, y)$.
2) $\exists x \to y P(x, y)$.
3) $\exists x \forall y P(x, y) \to Q(z)$.
4) $(\neg A \to B) \to (A \to B)$.
5) $a \to \forall y P(x, y)$.
6) $\exists x \forall y \big(P(x, y) \to Q(x)\big)$.
7) $\forall x P(x) \to \forall y Q(y)$.
8) $\neg\big(\exists x \forall z \big(P(x, y) \to P(y, z)\big)\big)$.
9) $\big(P(x) \to Q(x)\big) \wedge \exists x \big(\forall x R(x)\big)$.
10) $\forall x \big(P(x) \to Q(x)\big) \to \forall x Q(x, y)$.

2. Пусть n – натуральное число. Даны следующие утверждения.

- $A(n)$ – «число n кратно 5»;
- $B(n)$ – «число n кратно 2»;
- $C(n)$ – «число n кратно 4»;
- $D(n)$ – «число n кратно 10»;
- $E(n)$ – «число n кратно 20».

Укажите, какие из следующих высказываний истинны, какие ложны.

1) $\forall n D(n)$.
2) $\exists n E(n)$.
3) $\forall n \big(D(n) \to A(n) B(n)\big)$.
4) $\exists n \big(A(n) B(n) \to C(n)\big)$.

5) $\forall n\left(\neg A(n) \to \neg E(n)\right)$.

6) $\exists n\left(A(n)B(n)\right)$.

7) $\forall n\left(A(n)B(n)\right)$.

8) $\forall n\left(A(n) \to B(n)\right)$.

9) $\forall n\left(B(n)D(n) \to E(n)\right)$.

10) $\exists n\left(B(n)C(n) \to \neg D(n)\right)$.

3. Записать утверждения с помощью следующих обозначений: x, y – человек, i – преподаватель Иванов, $S(x)$ – студент, $Sc(x)$ – школьник, $E(x)$ – отличник, $C(x)$ – староста, $T(x)$ – преподаватель, $W(x)$ – работающий, $P(x)$ – член профсоюза, $Y(x)$ – молодой, $O(x)$ – старый, $J(x)$ – справедливый, $G(x)$ – девушка, $A(x,y)$ – x боится y.

1) Некоторые школьники и студенты – отличники.
2) Все старосты отличники и работают.
3) Все преподаватели и студенты являются членами профсоюза.
4) Не все молодые преподаватели справедливы.
5) Некоторые молодые и все старые преподаватели справедливы.
6) Все студенты и некоторые преподаватели молоды.
7) Среди работающих студентов есть отличники.
8) Некоторые студенты боятся преподавателя Иванова.
9) Никто из студенток не боится преподавателя Иванова.
10) Среди студенток-старост есть отличницы.

4. Построить систему собственных аксиом для следующих систем:

1) Линейное векторное пространство.
2) Группа (алгебраическая).
3) Метрическое пространство.
4) Семья.
5) Студенческая группа.

Глава 7. АЛГОРИТМЫ

Понятие алгоритма в настоящее время широко используется, тем не менее, строго это понятие не определено.

В математике алгоритм означает точное предписание, которое задает вычислительный процесс, начинающийся с исходных данных и направленный на получение полностью определенного исходными данными результата (см., например, [9]).

Примерами алгоритмов являются:

- Правила выполнения сложения и умножения натуральных чисел. Исходными данными являются пары натуральных чисел, результатом – натуральное число.
- Правило отыскания корней квадратного уравнения с действительными коэффициентами. Исходными данными являются коэффициенты уравнения $ax^2 + bx + c = 0$, результатом – два (может быть, равных) корня квадратного уравнения (если результат ищется в комплексных числах). Если результат обязательно должен быть действительным числом, то при $D = b^2 - 4ac < 0$ результат получен не будет.
- Правило отыскания производной многочлена степени *n*. Исходными данными являются коэффициенты многочлена, результатом – коэффициенты производной многочлена.

Алгоритм может оперировать не только с числами, но и с любыми символами и их последовательностями (словами). Следовательно, алгоритм можно понимать более широко, как тот или иной способ или путь решения некоторой задачи. Примерами могут быть: алгоритм расстановки мебели в квартире, алгоритм строительства дома и т.д.

Алгоритм должен удовлетворять следующим требованиям.

- *Массовость* алгоритма. Алгоритм применяется к исходным данным, которые выбираются из потенциально бесконечного множества.
- *Дискретность* алгоритма. Для размещения данных необходима однородная и дискретная, то есть организованная в виде одинакового количества ячеек, память.
- *Элементарность* шагов алгоритма. Закон получения последующей системы величин из предшествующей системы должен быть простым.
- *Детерминированность* алгоритма. Величины, получаемые на каком-либо (не начальном) этапе решения задачи, должны однозначно определяться величинами, полученными на предшествующих этапах. Кроме того, после каждого шага алгоритма должно быть указано, какой шаг выполняется дальше, либо дается команда остановки.
- *Результативность* алгоритма. После конечного числа шагов работа должна быть остановлена с указанием того, что считать результатом.

Можно выделить семь параметров, характеризующих алгоритм:

1) совокупность возможных исходных данных;
2) совокупность возможных результатов;
3) совокупность возможных промежуточных результатов;
4) правило начала;
5) правило непосредственной переработки;
6) правило окончания;
7) правило извлечения результата.

Мы рассмотрим элементы теории алгоритмов – рекурсивные функции и машины Тьюринга.

Глава 8. РЕКУРСИВНЫЕ ФУНКЦИИ

Рекурсивные функции очень хорошо иллюстрируют понятие алгоритма. Если рассуждать упрощенно, то для рекурсивной функции должен существовать алгоритм, вычисляющий ее значения. Вообще говоря, большая часть известных числовых функций являются рекурсивными.

Полезно вспомнить, как определяются *элементарные функции*. Вначале рассматривается несколько классов функций: алгебраические, тригонометрические, показательные, логарифмические. Элементарная функция определяется как *суперпозиция* (или сложная функция) этих функций.

Рекурсивные функции строятся аналогичным образом.

Обратите внимание, что все функции в данном параграфе определены на множестве $N \cup \{0\} = N_0$. Если это необходимо, в обозначении функции верхний индекс указывает число переменных. Так, функция $f^n(x_1, x_2, ..., x_n)$ зависит от n переменных. Таким образом, $f^n : N_0^n \to N_0$.

Рассмотрим вначале примитивно-рекурсивные функции.

Простейшие примитивно-рекурсивные функции задаются следующим образом.

- *Функция следования* задается формулой: $s(x) = x + 1$ (или $s^1(x) = x + 1$).
- *Функция аннулирования* задается формулой: $0(x) = 0$.
- *Функция тождества* определяется следующим образом:

  $I_i^n(x_1, x_2, ..., x_i, ..., x_n) = x_i, 1 \le i \le n$, то есть эта функция произвольному n-мерному вектору сопоставляет его i-ю координату.

Из простейших примитивно-рекурсивных функций можно получить примитивно-рекурсивные функции с помощью следующих двух операторов.

- *Оператор суперпозиции.* Пусть $f^n(x_1, x_2, ..., x_n)$, $g_1^m(t_1, t_2, ..., t_m)$, $g_2^m(t_1, t_2, ..., t_m)$, …, $g_n^m(t_1, t_2, ..., t_m)$ – примитивно-рекурсивные функции. Тогда функция

$$F_n^m(g_1^m, g_2^m, ..., g_n^m) =$$
$$= f^n(g_1^m(t_1, t_2, ..., t_m), g_2^m(t_1, t_2, ..., t_m), ..., g_n^m(t_1, t_2, ..., t_m))$$

получена с помощью оператора суперпозиции.

Оператор суперпозиции – это оператор построения сложной функции. Если мы умеем вычислять функции g_1^m, g_2^m, …, g_n^m и f^n, то значения функции F_n^m могут быть получены последовательным вычислением значений функций g_1^m, g_2^m, …, g_n^m на некотором наборе значений $a_1, a_2, ..., a_m$ переменных $t_1, t_2, ..., t_m$, и вычислением значения функции f^n на наборе значений $g_1^m(a_1, a_2, ..., a_m)$, $g_2^m(a_1, a_2, ..., a_m)$, …, $g_n^m(a_1, a_2, ..., a_m)$.

Пример. Функция $f(x) = 1$ получается суперпозицией функций 0(*x*) и *s*(*x*): $f(x) = s(0(x))$. Аналогичным образом можно получить функции вида $f(x) = n$ для всех значений *n*.

- *Оператор примитивной рекурсии* из известных примитивно-рекурсивных функций $\varphi^n(x_1, x_2, ..., x_n)$ и $\psi^{n+2}(x_1, x_2, ..., x_n, y, z)$ позволяет строить новую функцию $f^{n+1}(x_1, x_2, ..., x_n, y)$. Так,

$$f^{n+1}(x_1, x_2, ..., x_n, 0) = \varphi^n(x_1, x_2, ..., x_n),$$
$$f^{n+1}(x_1, x_2, ..., x_n, y+1) =$$
$$= \psi^{n+2}(x_1, x_2, ..., x_n, y, f^{n+1}(x_1, x_2, ..., x_n, y)).$$

Тогда функция $f^{n+1}(x_1, x_2, ..., x_n, y)$ получена с помощью оператора примитивной рекурсии, что выражается обозначением $f^{n+1} = R_n(\varphi^n, \psi^{n+2})$.

Таким образом, сначала (при фиксированных значениях $x_1, x_2, ..., x_n$) определяется значение функции f^{n+1} при $y = 0$, а затем каждое следующее значение функции (зависящее от $y+1$) выражается через предыдущее значение (зависящее от y).

Пусть $n = 0$. Тогда функция φ^0 есть постоянная. Обозначим ее следующим образом: $\varphi^0 = a$. Функция ψ^{n+2} зависит от двух переменных. Обозначим ее так: $\psi^2(y, z) = \psi(y, z)$. Тогда

$f(0) = a$,

$f(y+1) = \psi(y, f(y))$.

Для произвольного n получаем (обозначения b_0, b_1, b_2, …, b_y вводятся в предположении, что набор $x_1, x_2, ..., x_n$ фиксирован):

$f^{n+1}(x_1, x_2, ..., x_n, 0) = \varphi^n(x_1, x_2, ..., x_n) = b_0$,

$f^{n+1}(x_1, x_2, ..., x_n, 1) = \psi^{n+2}(x_1, x_2, ..., x_n, 0, b_0) = b_1$,

$f^{n+1}(x_1, x_2, ..., x_n, 2) = \psi^{n+2}(x_1, x_2, ..., x_n, 1, b_1) = b_2$,

. . . . . .

$f^{n+1}(x_1, x_2, ..., x_n, y+1) = \psi^{n+2}(x_1, x_2, ..., x_n, y, b_y)$.

. . . . . .

Пример. Даны функции $\varphi(x) = x$ и $\psi(x, y, z) = xz$. Определим функцию $f(x, y)$, полученную из данных функций по схеме примитивной рекурсии.

Решение. Найдем значения функции $f(x, y)$.

$f(x,0) = \varphi(x) = x$,

$f(x,1) = \psi(x,0, f(x,0)) = \psi(x,0,x) = x \cdot x = x^2$;

$f(x,2) = \psi(x,1, f(x,1)) = \psi(x,0,x^2) = x \cdot x^2 = x^3$;

$f(x,3) = \psi(x,2, f(x,2)) = \psi(x,0,x^3) = x \cdot x^3 = x^4$.

Можно предположить, что $f(x, y) = x^{y+1}$.

Докажем последнюю формулу методом математической индукции по переменной y.

1. Проверим формулу при $y = 0$.

$f(x,0) = x = x^{0+1}$, то есть при $y = 0$ формула верна.

2. Допустим, что предположение индукции верно при $y = n$, то есть, верна формула $f(x, n) = x^{n+1}$.

Докажем, что предположение индукции верно при $y = n+1$, то есть, верна формула $f(x, n+1) = x^{n+2}$. Выразим $f(x, n+1)$ с помощью схемы примитивной рекурсии.

$f(x, n+1) = \psi(x, n, f(x, n)) = \psi(x, n, x^{n+1}) = x \cdot x^{n+1} = x^{n+2}$.

Таким образом, на основании метода математической индукции формула $f(x, y) = x^{y+1}$ доказана для всех $y \in N_0$.

Теперь строго определим примитивно-рекурсивные функции.

Определение. 1) Простейшие примитивно-рекурсивные функции примитивно-рекурсивны.

2) Примитивно-рекурсивными являются функции, полученные из примитивно-рекурсивных функций с помощью операторов суперпозиции и (или) примитивной рекурсии.
3) Функция является примитивно-рекурсивной тогда и только тогда, когда это следует из 1) и 2).

Пример. Покажем, что функция $f_+(x, y) = x + y$ примитивно-рекурсивна.

Доказательство. $n+1 = 2$, $n = 1$, следовательно, функция φ должна зависеть от одной переменной, а функция ψ – от трех. Пользуясь заданием функции, найдем ее значения:

$f_+(x, 0) = x = I_1^1(x) = \varphi(x)$.

$f_+(x, y+1) = x + y + 1 = f_+(x, y) + 1 = s^1\left(f_+(x, y)\right) =$

$= \psi\left(x, y, f_+(x, y)\right)$,

то есть $\psi\left(x, y, z\right) = s(z) = s\left(I_3^3(x, y, z)\right)$.

Таким образом, функция $f_+(x, y) = x + y$ получена по схеме примитивной рекурсии ($f_+ = R_1(\varphi, \psi)$) из примитивно-рекурсивных функций, следовательно, сама является примитивно-рекурсивной.

Примитивно-рекурсивными, в частности, являются следующие функции: $f(x) = a$,

$f(x) = x + a$, $f(x, y) = x + y$, $f(x, y) = xy$, $f(x, y) = x^y$, $f(x, y) = x!$,

$\operatorname{sgn}(x) = \begin{cases} 0, \text{ если } x = 0, \\ 1, \text{ если } x > 0, \end{cases}$ $\overline{\operatorname{sgn}}(x) = \begin{cases} 1, \text{ если } x = 0, \\ 0, \text{ если } x > 0. \end{cases}$

Операция минимизации по i-ой переменной функции $f^n\left(x_1, x_2, \ldots, x_n\right)$ обозначается следующим образом: $\mu_y\left(f^n\left(x_1, x_2, \ldots, x_{i-1}, y, x_{i+1}, \ldots, x_n\right) = x_i\right)$, и определяется так.

Рассмотрим уравнение относительно y:

$$f^n(x_1, x_2, ..., x_{i-1}, y, x_{i+1}, ..., x_n) = x_i. \quad (1)$$

Это уравнение решается подбором, вместо переменной y последовательно подставляются 0,1,2,… При этом возможны случаи.

- На некотором шаге левая часть соотношения (1) не определена. Следовательно, на наборе $(x_1, x_2, ..., x_i, ..., x_n)$ операция минимизации не определена.
- На каждом шаге левая часть соотношения (1) определена, но равенство не выполняется ни при каких значениях y. Следовательно, на наборе $(x_1, x_2, ..., x_i, ..., x_n)$ операция минимизации не определена.
- Левая часть соотношения (1) определена при $y \le z$, но при $y < z$ равенство не выполняется, а при $y = z$ выполняется. В этом случае число z считается значением операции минимизации на наборе $(x_1, x_2, ..., x_n)$.

Пример. [13]. Найти функции, получаемые из данной числовой функции $f(x_1, x_2, x_3) = 1 - \frac{x_1}{x_2}$ с помощью оператора минимизации по каждой ее переменной.

Решение. Минимизируем функцию по переменной x_1. Рассмотрим уравнение

$$1 - \frac{y}{x_2} = x_1. \quad (2)$$

1. Если $x_1 = 1$, $x_2 \neq 0$, то при подстановке $y = 0$ получаем верное равенство.
2. Если $x_2 = 0$, то левая часть равенства (2) не определена.
3. Если $x_1 \neq 1$, $x_2 \neq 0$, то при подстановке $y = 1$ в левой части равенства (2) появляется выражение $\frac{1}{x_2}$, не имеющее смысла, и в этом случае операция минимизации не определена.
4. Если $x_2 = 1$, то получаем равенство $1 - y = x_1$. Оно имеет смысл при $x_1 = 1$, то есть $y = 0$, что рассмотрено в первом пункте, и при $x_1 = 0$, то есть $y = 1$. При $x_1 > 1$ равенство не имеет смысла.

Таким образом,

$$\mu_y\left(1 - \frac{y}{x_2} = x_1\right) = \begin{cases} 0, \text{ если } x_1 = 1, x_2 \neq 0; \\ \text{не определено, если } x_2 = 0; \\ \text{не определено, если } x_1 > 1, x_2 \neq 0; \\ x_2, \text{ если } x_1 = 0. \end{cases}$$

Минимизируем функцию по переменной x_2. Рассмотрим уравнение

$$1-\frac{x_1}{y}=x_2.$$

Это уравнение на самом первом шаге, при подстановке вместо y нуля теряет смысл, значит, операция минимизации по второй переменной $\mu_y\left(1-\frac{x_1}{y}=x_2\right)$ нигде не определена.

Минимизируем функцию по переменной x_3. Рассмотрим уравнение

$$1-\frac{x_1}{x_2}=x_3. \qquad (3)$$

Если левая часть соотношения (3) имеет смысл и равенство (3) выполнено, то оно выполнено и при подстановке в это соотношение переменной y на первом шаге, то есть при $y=0$. В остальных случаях значение операции минимизации не определено.

$$\mu_y\left(1-\frac{x_1}{x_2}=x_3\right)=\begin{cases}0, \text{ если } 1-\frac{x_1}{x_2}=x_3;\\ \text{не определено в остальных случаях.}\end{cases}$$

Определение. *Частично-рекурсивной функцией* называется числовая функция, получаемая за конечное число шагов из простейших примитивно-рекурсивных функций с помощью операторов суперпозиции, примитивной рекурсии и минимизации.

Определение. Числовая функция называется *общерекурсивной*, если она частично-рекурсивна и всюду определена.

Определение. Функция $f\left(x_1,x_2,...,x_n\right)$ называется *эффективно вычислимой*, если существует алгоритм, позволяющий вычислить ее значения.

В данном определении алгоритм понимается в интуитивном значении, следовательно, интуитивным является и понятие эффективно вычислимой функции.

Имеет место следующий тезис.

Тезис Черча. Каждая интуитивно вычислимая функция является частично-рекурсивной.

Тезис является недоказуемым, так как он связывает нестрогое понятие интуитивно вычислимой функции и строгое математическое понятие частично-рекурсивной функции.

Тезис может быть опровергнут построением примера интуитивно вычислимой, но не частично-рекурсивной функции.

Задачи

1. Определить функцию $f(x,y)$, полученную из функций $\varphi(x)$ и $\psi(x,y,z)$ по схеме примитивной рекурсии.

1) $\varphi(x)=x$, $\psi(x,y,z)=z$.

2) $\varphi(x)=x$, $\psi(x,y,z)=x+z$.

3) $\varphi(x)=x$, $\psi(x,y,z)=x+y-z$.

4) $\varphi(x)=x$, $\psi(x,y,z)=z^2$.

5) $\varphi(x)=1$, $\psi(x,y,z)=xz$.

6) $\varphi(x)=1$, $\psi(x,y,z)=xy$.

7) $\varphi(x)=0$, $\psi(x,y,z)=x^2+z$.

8) $\varphi(x)=1$, $\psi(x,y,z)=\frac{x}{z}$.

9) $\varphi(x)=0$, $\psi(x,y,z)=|z-x|$.

10) $\varphi(x)=0$, $\psi(x,y,z)=x+y$.

2. Доказать, что следующие функции примитивно-рекурсивны.

1) $f(x)=a$.

2) $f(x)=x+a$.

3) $\operatorname{sgn}(x)=\begin{cases}0, \text{ если } x=0,\\ 1, \text{ если } x>0.\end{cases}$

4) $f(x,y)=xy$.

5) $f(x,y)=x!$.

6) $\overline{\operatorname{sgn}}(x)=\begin{cases}1, \text{ если } x=0,\\ 0, \text{ если } x>0.\end{cases}$

7) $f(x,y)=x^y$.

3. Записать схему примитивной рекурсии для произвольных примитивно-рекурсивных функций при

1) $n=1$;

2) $n=2$;

3) $n=3$.

4. Найти функции, получаемые из данной числовой функции $f(x_1, x_2, x_3)$ с помощью оператора минимизации по каждой ее переменной.

1) $f(x_1, x_2, x_3) = x_1^{\,2} - x_2$.
2) $f(x_1, x_2, x_3) = 2^{x_1}(x_2 + 1)$.
3) $f(x_1, x_2, x_3) = x_1 - x_2$.
4) $f(x_1, x_2, x_3) = x_1 x_2^{\,2} + 1$.
5) $f(x_1, x_2, x_3) = x_1^3 + x_2$.
6) $f(x_1, x_2, x_3) = x_1^3 + 3x_2$.
7) $f(x_1, x_2, x_3) = 2^{x_1} + \sqrt{x_2}$.
8) $f(x_1, x_2, x_3) = 2(x_2 - x_1)$.
9) $f(x_1, x_2, x_3) = x_1^{x_2}$.
10) $f(x_1, x_2, x_3) = x_1(x_2 - 1)^2$.

Глава 9. МАШИНЫ ТЬЮРИНГА

Машина Тьюринга – это модель алгоритма, которая иллюстрирует процессы, происходящие при реализации алгоритма. Машина Тьюринга является гипотетической машиной. Ее составляют следующие компоненты.

- *Управляющее устройство*, которое в каждый данный момент времени может находиться в одном и только одном из некоторого множества *состояний*. Состояния обозначаются буквами так называемого *внутреннего алфавита* $Q = \{q_0, q_1, \ldots, q_m\}$. Состояние q_1, как правило, считают *начальным состоянием*, а состояние q_0 – *конечным* (*заключительным*). Во внутренний алфавит включают также *символы сдвига* : R – вправо, L – влево, E – на месте.
- *Лента*, разделенная на ячейки и предполающаяся потенциально бесконечной в обе стороны (имеется в виду, что в каждый момент времени лента содержит конечное число ячеек, но при необходимости число ячеек можно увеличивать). В каждой ячейке может быть записан один и только один символ некоторого *внешнего алфавита* $A = \{a_0, a_1, \ldots, a_n\}$. Символ a_0 принято считать *пустым символом*. Он обозначает пустую ячейку. По умолчанию, во всех ячейках, в которых не записаны символы a_1, a_2,..., a_n, записан символ a_0. В дан-

ной главе в качестве внешнего алфавита мы будем рассматривать алфавит $E = \{0,1\}$, 0 соответствует пустому символу.

- *Считывающая и пишущая головка*, которая в каждый данный момент времени обозревает одну ячейку.

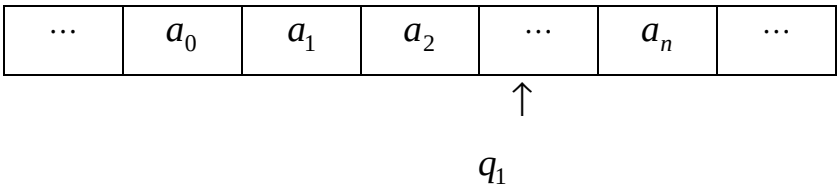


Рис. 1

Так, на рис. 1 считывающая головка обозревает ячейку ленты, в которой записан символ a_n. Управляющее устройство находится в состоянии q_1 (начальном состоянии). В зависимости от состояния управляющего устройства головка либо оставляет обозреваемый символ без изменения, либо записывает на его место любой другой символ внешнего алфавита, либо стирает обозреваемый символ. Далее головка либо остается на месте, либо передвигается на одну ячейку вправо или влево, при этом управляющее устройство переходит в некоторое новое состояние (состояние может и не меняться).

Каждое перемещение головки и изменение состояния управляющего устройства можно определить *командой*, которая обычно записывается в виде:

$q_i a_j q_{ij} a_{ij} D_{ij}$.

Здесь:

- q_i – состояние, в котором управляющее устройство находится в данный момент,
- a_j – символ, обозреваемый головкой,
- q_{ij} – состояние, в которое управляющее устройство переходит в зависимости от состояния q_i и обозреваемого символа a_j,
- a_{ij} – новый символ, который записывается в ячейку, и зависящий от q_i и a_j,
- D_{ij} – символ сдвига, указывающий направление движения головки (он также зависит от q_i и a_j).

Список команд для машины Тьюринга называется программой. Существует взаимно однозначное соответствие между машинами Тьюринга и программами.

Вид ленты в каждый момент времени может быть определен *конфигурацией* вида:

$a_{j_1} \ldots a_{j_{l-1}} q_i a_{j_l} \ldots a_{j_s}$.

Головкой в данный момент обозревается символ a_{j_l}, записанный в конфигурации первым справа от символа q_i. Первый и последний символы в данной конфигурации – непустые. Считается, что остальные символы на ленте, не записанные в конфигурацию – пустые. Имеется в виду, что в данный момент времени на ленте записано *слово* $a_{j_1} \ldots a_{j_{l-1}} a_{j_l} \ldots a_{j_s}$.

Конфигурация, соответствующая началу работы машины, называется начальной. Если в процессе работы машина достигает заключительного состояния, то соответствующая конфигурация называется заключительной. Машина может прекратить работу и в случае, когда в программе отсутствует команда для некоторого состояния и некоторого символа.

Если машина Тьюринга T, начав работу на некотором слове P, останавливается через некоторое число шагов, то считается, что она применима к слову P. Результатом применения машины к слову является слово $T(P)$, которое соответствует заключительной конфигурации. Если же машина, начав работу на слове P, никогда не останавливается, то говорят, что она не применима к слову P.

Пример. Пусть машина Тьюринга задана программой:

$$T: \begin{cases} q_1 0 q_0 1 E \\ q_1 1 q_1 1 R \end{cases}.$$

Рассмотрим записанные в последовательных *n* ячейках *n* единиц.

Пусть, например, начальная конфигурация имеет вид:

$q_1 111 \ldots 11$.

Сокращенно эту конфигурацию можно записать так:

$q_1 1^n$.

Вообще, записанные подряд *n* единиц обозначаются 1^n, а записанные подряд *m* нулей – 0^m.

Тогда в каждом такте машина Тьюринга будет оставлять обозреваемую единицу на месте и сдвигаться вправо на одну ячейку. Процесс будет продолжаться до тех пор, пока управляющая головка не выйдет на пустую ячейку (0). Здесь, согласно программе, в ячейку будет вписана единица, и машина остановится. В результате на ленте будет записано *n*+1 единиц. Если условиться, что исходное слово выражает число *n*, то можно считать, что машина вычисляет функцию $\varphi(n) = n + 1$.

Пример. Дана машина Тьюринга:

$$T: \begin{cases} q_1 0 q_1 0 R \\ q_1 1 q_2 0 R \\ q_2 1 q_1 0 R \end{cases}.$$

Выяснить, применима ли машина к слову P:

а) $P = 1^3 01$; б) $P = 1^6$.

Если применима, то выписать результат $T(P)$ применения машины T к слову P. Предполагается, что в начальный момент времени головка машины обозревает самую левую единицу слова.

Решение. а) Применяя машину T к слову P, получаем последовательность конфигураций:

1) $q_1 1^3 01$.
2) $q_2 1^2 01$.
3) $q_1 101$.
4) $q_2 01$.

Вид второй конфигурации обусловлен тем, что символ 0 считается пустым символом и может не записываться.

Поскольку команда вида $q_2 0 q_i \alpha D$ в программе отсутствует, то последняя конфигурация является заключительной. Следовательно, машина T к слову P применима, и $T(P) = 1$. (Нули слева и справа от слова не записываются).

б) Получаем конфигурации:

1) $q_1 1^6$.
2) $q_2 1^5$.
3) $q_1 1^4$.
4) $q_2 1^3$.
5) $q_1 1^2$.
6) $q_2 1$.
7) $q_1 0$.
8) $q_1 0$.
9) $q_1 0$.
. . . . . .

Процесс продолжается неограниченно, головка смещается по ленте вправо до бесконечности, следовательно, машина T к слову $P = 1^6$ не применима.

Вид конфигурации 8) обусловлен тем, что символ 0 (пустой символ) находится справа от последней единицы слова по умолчанию.

Машины Тьюринга T_1 и T_2 называются эквивалентными, если:

- T_1 и T_2 либо обе применимы, либо обе не применимы к каждому исходному слову P;
- если обе машины применимы к слову P, то $T_1(P) = T_2(P)$.

Программу для машины Тьюринга можно задать не только с помощью последовательности команд, но и в виде таблицы. Так, в последнем примере программа может быть задана в виде:

	q_1	q_2
0	$q_1 0 R$	–
1	$q_2 0 R$	$q_1 0 R$

При табличной записи командой иногда называют выражение $q_{ij}a_{ij}D_{ij}$.

Имеет место следующий тезис.

Тезис Тьюринга. Всякий алгоритм может быть реализован соответствующей машиной Тьюринга.

Тезис является недоказуемым, так как он связывает нестрогое понятие алгоритма и строгое понятие машины Тьюринга.

Тезис может быть опровергнут построением примера алгоритма, который не может быть реализован машиной Тьюринга.

Операции с машинами Тьюринга

Очевидно, что некоторые алгоритмы могут быть составлены из нескольких более простых алгоритмов, и наоборот, могут служить основой для построения новых алгоритмов.

Точно также, удобно строить машины Тьюринга, исходя из уже построенных машин.

Принцип двойственности

Пусть T – произвольная программа (машина Тьюринга). Обозначим T^* программу, которая получается из T заменой (во всех командах) R на L и наоборот. Программа T^* называется *двойственной* к T.

Пример. Машина Тьюринга в произвольной записи, начиная с любой ячейки, двигаясь вправо, находит первый нуль. Соответствующая программа имеет вид:

$$T:\begin{cases} q_1 0 q_0 0 E \\ q_1 1 q_1 1 R \end{cases}.$$

Возможны три случая.

1. В начальный момент головка машины обозревает нуль. Машина останавливается.

2. В начальный момент головка машины обозревает единицу, и справа от начальной ячейки есть хотя бы один нуль. Машина переместит головку через массив единиц вправо и остановится перед первым нулем.

3. В начальный момент головка машины обозревает единицу, и справа от начальной ячейки запись состоит только из единиц. Машина будет перемещать головку через массив единиц вправо, не останавливаясь.

В программе заменим символ R на L. Получим программу:

$$T:\begin{cases} q_1 0 q_0 0 E \\ q_1 1 q_1 1 L \end{cases}.$$

Данная программа будет двойственной к предыдущей. Непосредственной проверкой можно убедиться, что головка машины, двигаясь влево, будет отыскивать первый нуль.

Очевидно, что (*T**)*=*T*, то есть понятие двойственности является взаимным. Машины Тьюринга, соответствующие двойственным программам, будем называть двойственными машинами Тьюринга.

Из примера было видно, что двойственные машины функционируют симметричным образом. Так, пусть в начальный момент времени имеется конфигурация

$$\ldots q_i a_1 a_2 \ldots,$$

и машина *T* в момент времени *t* переработает ее в конфигурацию

$$\ldots c_1 c_2 \ldots q_i c_s \ldots.$$

В то же время, двойственная машина *T** конфигурацию

$$\ldots a_2 q_i a_1 \ldots$$

(симметричную первой конфигурации относительно a_1) в момент времени *t* переработает в конфигурацию

$$\ldots q_i c_s \ldots c_2 c_1 \ldots,$$

симметричную второй конфигурации относительно c_1.

Способы композиции машин Тьюринга

1. *Последовательное подключение одной машины Тьюринга к другой.* Пусть T_0 и T_1 – две машины Тьюринга над алфавитом {0,1}, множества состояний машин не пересекаются. Перенумеруем 0,1,…,*l*-1 все команды с q_0 программы T_0. Пусть *p*(*x*) – предикат на множестве {0,1,…,*l*-1}. *Последовательное подключение* T_1 *к* T_0 (относительно предиката *p*(*x*)) – это машина Тьюринга T, которая получается следующим образом. Первая половина таблицы для T совпадает с таблицей T_0 для тех клеток, в которых нет команды с q_0.

Если *p*(*n*)=1, то в клетке *n* – команда $q_1' a E$, a – номер строки (0 или 1), где находится клетка *n*, q_1' – начальное состояние T_1.

Если *p*(*n*)=0, то в клетке *n* – команда с q_0. Вторая половина таблицы *T* полностью совпадает с таблицей T_1.

	$q_1 \ldots q_j \ldots q_m$	$q_1' \ldots q_j' \ldots q_m'$
0		
1		

В частном случае, если q_0 – начальное состояние машины T_0, а q_1' – начальное состояние T_1, заменим в программе T_0 состояние q_0 на состояние q_1', и полученную программу объединим с программой T_1. В результате мы получим программу для машины T, которая является композицией машин T_0 и T_1 по паре состояний (q_0, q_1').

2. *Итерация машины Тьюринга.* Пусть T_0 – машина Тьюринга над алфавитом {0,1}. Перенумеруем 0,1,…,*l*-1 все команды с q_0 программы T_0. Пусть *p*(*x*) – предикат на множестве {0,1,…,*l*-1}. Итерация машины Тьюринга T_0 относительно предиката *p*(*x*) – это машина Тьюринга *T*, которая получается следующим образом. Таблица *T* совпадает с таблицей T_0 для тех клеток, в которых нет команды не с q_0.

Если *p*(*n*)=1, то в клетке *n* – команда q_1aE, *a* – номер строки, где находится клетка *n*, q_1 – начальное состояние T_0.

Если *p*(*n*)=0, то в клетке *n* – команда с q_0.

Действительно, имеет место итерация, т.е. многократная работа машины T_0.

В частном случае, если q_0 – заключительное состояние машины T_0, а q' – любое состояние машины T_0, не являющееся заключительным, то заменим в программе T_0 состояние q_0 на состояние q'. В результате мы получим программу для машины *T*, которая является итерацией машины T_0 по паре состояний (q_0, q').

Отметим, что начальных и заключительных состояний может быть несколько.

Задачи

1. По заданной машине Тьюринга T и начальной конфигурации K_1 найти заключительную конфигурацию:

1) $T: \begin{cases} q_1 0 q_1 1 R \\ q_1 1 q_0 1 E \end{cases}$; $K_1 = 1 q_1 01^3$.

2) $T: \begin{cases} q_1 0 q_1 1 R \\ q_1 1 q_2 1 L \\ q_2 0 q_0 0 R \\ q_2 1 q_1 0 R \end{cases}$; $K_1 = q_1 101^3$.

3) $T:\begin{cases} q_1 0 q_2 1 R \\ q_1 1 q_2 0 L \\ q_2 0 q_0 1 E \\ q_2 1 q_1 1 L \end{cases}$; $K_1 = 10^2 1 q_1 1$.

4) $T:\begin{cases} q_1 0 q_2 0 R \\ q_1 1 q_2 1 L \\ q_2 0 q_0 0 E \\ q_2 1 q_1 1 R \end{cases}$; $K_1 = 101^2 q_1 1^2$.

5) $T:\begin{cases} q_1 0 q_1 0 R \\ q_1 1 q_2 1 L \\ q_2 0 q_0 1 E \\ q_2 1 q_2 1 R \end{cases}$; $K_1 = 10 q_1 01$.

6) $T:\begin{cases} q_1 0 q_2 1 R \\ q_1 1 q_1 1 L \\ q_2 0 q_2 0 L \\ q_2 1 q_0 1 L \end{cases}$; $K_1 = 1^2 0 q_1 1$.

7) $T:\begin{cases} q_1 0 q_2 0 R \\ q_1 1 q_1 1 L \\ q_2 0 q_0 1 E \\ q_2 1 q_1 0 L \end{cases}$; $K_1 = 1^2 q_1 01^3$.

8) $T:\begin{cases} q_1 0 q_0 1 E \\ q_1 1 q_2 0 R \\ q_2 0 q_1 0 R \end{cases}$; $K_1 = 1^2 q_1 101$.

9) $T:\begin{cases} q_1 0 q_0 0 E \\ q_1 1 q_2 1 L \\ q_2 0 q_0 1 R \\ q_2 1 q_3 0 L \\ q_3 0 q_1 1 R \\ q_3 1 q_1 0 L \end{cases}$; $K_1 = 1^5 q_1 1$.

10) $T:\begin{cases} q_1 0 q_0 0 E \\ q_1 1 q_2 1 R \\ q_2 0 q_0 1 L \\ q_2 1 q_3 0 R \\ q_3 0 q_1 1 L \\ q_3 1 q_1 0 R \end{cases}$; $K_1 = q_1 1^3 01$.

2. Выяснить, применима ли машина Тьюринга T к слову P. Если применима, то записать результат $T(P)$ применения машины T к слову P. Предполагается, что в начальный момент времени головка машины обозревает самую левую единицу слова.

1) $T:\begin{cases} q_1 0 q_1 0 R \\ q_1 1 q_2 1 L \\ q_2 0 q_0 0 E \\ q_2 1 q_1 1 R \end{cases}$; а) $P = 1^3 01$; б) $P = 101$.

2) $T:\begin{cases} q_1 0 q_2 0 R \\ q_1 1 q_2 0 R \\ q_2 1 q_1 1 R \end{cases}$; а) $P = 1^3$; б) $P = 10^2 1$.

3) $T:\begin{cases} q_1 0 q_1 1 L \\ q_1 1 q_2 0 L \\ q_2 0 q_1 1 L \\ q_2 1 q_0 1 E \end{cases}$; а) $P = 10^2 1$; б) $P = 1^2 0^2 1^2$.

4) $T:\begin{cases} q_1 0 q_2 1 R \\ q_1 1 q_1 0 L \\ q_2 0 q_3 1 R \\ q_2 1 q_3 0 L \\ q_3 0 q_1 0 R \end{cases}$; а) $P = 10^3 1$; б) $P = [10]^2 1$.

5) $T:\begin{cases} q_1 0 q_2 1 R \\ q_1 1 q_2 1 R \\ q_2 0 q_3 0 R \\ q_2 1 q_1 0 L \\ q_3 0 q_2 1 E \\ q_3 1 q_0 0 L \end{cases}$; а) $P = 1^2$; б) $P = 10^4 1$.

3. Построить в алфавите {0,1} машину Тьюринга, обладающую свойствами:

1) машина имеет одно состояние, одну команду и применима к любому слову в алфавите {0,1};
2) машина имеет одно состояние, две команды, не применима ни к какому слову в алфавите {0,1}, и в процессе работы головка обозревает бесконечное множество ячеек;
3) машина имеет две команды, не применима ни к какому слову в алфавите {0,1}, и в процессе работы головка обозревает одну ячейку.

Предполагается, что в начальный момент времени головка машины обозревает самый левый символ слова.

4. По словесному описанию машины Тьюринга построить ее программу (в алфавите {0,1}).

1) Начав работу с первой единицы массива из единиц, машина «сдвигает» его на две ячейки вправо, не изменяя остального содержимого ленты, и останавливается на последней единице перенесенного массива.
2) Начав двигаться влево от произвольной ячейки, головка находит первую при таком перемещении ячейку с единицей (если такая встретится на пути) и, сделав один шаг вправо, останавливается на соседней ячейке. Содержимое ленты не меняется.
3) Машина начинает работу с самой левой непустой ячейки и отыскивает нуль, примыкающий с левой стороны к первому справа массиву из трех единиц, окаймленному нулями. Головка останавливается на первой единице найденного массива (если такой есть). Содержимое ленты не меняется.
4) Головка машины, начав работу с произвольной ячейки, содержащей единицу, двигается влево до тех пор, пока не пройдет подряд пять нулей. Головка останавливается на первой ячейке слева за этими пятью нулями, напечатав в ней единицу. Остальное содержимое ленты не меняется.
5) При заданном $n \geq 1$ головка машины, начав работу с произвольной ячейки и двигаясь вправо, записывает подряд $2n$ нулей и останавливается на последнем из них.
6) Головка машины, двигаясь вправо от какой-либо пустой ячейки, находит первый при таком перемещении массив, содержащий не менее семи единиц, стирает в нем первые семь единиц и останавливается на самой правой из ячеек, в которых были стерты единицы. Остальное содержимое ленты не меняется.
7) При заданном значении *n* головка машины из *n* записанных единиц оставляет на ленте $n-2$ единицы, так же записанных подряд, если $n \geq 2$, и работает вечно, если $n = 0$ или $n = 1$.
8) Машина реализует алгоритм вычисления функции $\varphi(n) = 0$, считая, что число *n* представляется записанными подряд *n* единицами, и массив из *n* единиц уже найден.

9) Машина реализует алгоритм вычисления функции $\varphi(n)=1$, считая, что число *n* представляется записанными подряд *n* единицами, и массив из *n* единиц уже найден.

10) Машина реализует алгоритм вычисления функции $f_p(n)=\begin{cases}1, \text{если } n \text{ делится на } p,\\ 0, \text{если } n \text{ не делится на } p.\end{cases}$

Считается, что число *n* представляется записанными подряд *n* единицами, и массив из *n* единиц уже найден.

11) Показать, что для всякой машины Тьюринга существует эквивалентная ей машина, в программе которой отсутствуют заключительные состояния.

12) Показать, что для всякой машины Тьюринга существует эквивалентная ей машина, в программе которой отсутствует символ *E*.

5. Для машин Тьюринга из задачи 1 построить двойственные машины.

6. Построить композицию T_1T_2 машин Тьюринга T_1 и T_2 по паре состояний (q_{10}, q_{21}) и найти результат применения композиции T_1T_2 к слову P.

1)

T_1:

	q_{11}	q_{12}
0	$q_{12}\,0R$	$q_{10}\,1L$
1	$q_{12}\,1L$	$q_{11}\,1R$

, T_2:

	q_{21}	q_{22}
0	$q_{22}\,1R$	$q_{21}\,1L$
1	$q_{21}\,1R$	$q_{20}\,1E$

а) $P=101$; б) $P=1^3$.

2)

T_1:

	q_{11}	q_{12}	q_{13}
0	$q_{10}\,0L$	$q_{13}\,0R$	$q_{11}\,0R$
1	$q_{12}\,1E$	$q_{13}\,1L$	$q_{11}\,0R$

T_2:

	q_{21}	q_{22}
0	$q_{22}\,1R$	$q_{20}\,0R$
1	$q_{22}1L$	$q_{21}\,0L$

а) $P=1^201$; б) $P=1^4$.

3)

T_1:

	q_{11}	q_{12}	q_{13}
0	$q_{12}\,0R$	$q_{13}\,0L$	$q_{10}\,1L$
1	$q_{11}\,1R$	$q_{13}\,1R$	–

T_2:

	q_{21}	q_{22}	q_{23}
0	$q_{22}\,0L$	$q_{23}\,0R$	$q_{20}\,0R$
1	$q_{21}\,1L$	$q_{22}\,1R$	$q_{21}\,1L$

а) $P=1^3$; б) $P=101^2$.

7. Найти результат применения итерации машины T по паре состояний $(q_0,\ q_1)$ к слову P (заключительными состояниями являются q_0 и q_0').

1)

T:

	q_1	q_2	q_3	q_4	q_5
0	$q_0 0E$	$q_4 0E$	$q_5 0E$	$q_3 1R$	$q_0 1R$
1	$q_2 0R$	$q_2 0R$	$q_1 0R$	–	–

а) $P = 1^3$; б) $P = 101$.

2)

T:

	q_1	q_2	q_3	q_4	q_5	q_6
0	$q_0' 0R$	$q_0' 0R$	$q_4 0R$	$q_5 1L$	$q_6 0L$	$q_0 0R$
1	$q_2 0R$	$q_3 0R$	$q_3 1R$	$q_4 1R$	$q_5 1L$	$q_6 1L$

а) $P = 1^2$; б) $P = 1^5$.

Ответы и указания

Глава 1. ВЫСКАЗЫВАНИЯ, ФОРМУЛЫ, ТАВТОЛОГИИ 2. 2), 5), 8), 10) – истинные высказывания. 1), 4), 7) – ложные высказывания. 3), 6) высказываниями не являются. 3. Обратите внимание, что 7) – составное высказывание. 5. Не являются тавтологиями: 2), 3), 5). Вернуться в **Задачи**

Глава 3. ИСЧИСЛЕНИЕ ВЫСКАЗЫВАНИЙ 1. 1), 2), 5), 7), 8), 9) – формулы исчисления высказываний. 3), 4), 6), 10) формулами исчисления высказываний не являются. 2. 2). 3. 5) Применить лемму. 6), 7), 9) – применить результат 5). 8), 10), 11), 12) – применить результат 9). Вернуться в **Задачи**

Глава 5. ПРЕДИКАТЫ 1. 1) $\{-2\}$. 3) $\{2\}$. 5) $\{(x,y)\in R^2 \mid x-y=1\}$. 7) R. 8) $\{(x,y)\in R^2 \mid x\ge y^2\}$. 9) $\{1,2,3,5,7\}$. 10) $\{-1\}$. 2. 1), 3), 4), 6), 7), 9), 10). 1. 2), 5), 8). 0. 3. 7) Воспользоваться формулой $A\leftrightarrow B = \neg A\neg B\vee AB$.

4. 1) $\forall x\forall t\forall z(\neg R(x,y,z)\vee P(t,y))$.

2) $\exists x\forall t(\neg R(x,t,z)\vee P(x,y))$. 3) $\forall t\exists s(P(t,y)\wedge Q(x,s))$.

4) $\exists x \forall t(P(x,t) \wedge Q(x,y))$. 5) $\exists t \exists s \forall z(\neg R(x,t) \vee P(s,y,z))$.

6) $\forall t \exists x \exists y \exists z(P(t,y) \wedge Q(x,z))$. 7) $\exists x \forall z(\neg P(x,y) \vee Q(x,z))$.

8) $\exists x \forall y \forall z(P(x,y) \vee \neg Q(x,z))$.

9) $\forall s \exists z \forall t(\neg P(s,y,z) \vee R(x,y,t) \vee Q(x,y))$.

10) $\exists x \forall t(\neg P(x,y,z) \vee Q(x,y,t))$. 8. Можно привести такой пример: $P(x) = "x = 0"$, $Q(x) = "x = 1"$. Предикаты рассматриваются на множестве R. Вернуться в **Задачи**

Глава 8. РЕКУРСИВНЫЕ ФУНКЦИИ 1. 1) x. 2) $x(y+1)$.

3) $f(x,y) = \begin{cases} x + \dfrac{y}{2}, y = 2k, \\ y - 1, y = 2k+1. \end{cases}$ 4) x^{2^y}. 5) x^y. 6) $\overline{\text{sgn}}(x)$. 7) $x^2 y$.

8) $f(x,y) = \begin{cases} 1, y = 2k, \\ x, y = 2k+1. \end{cases}$ 9) $f(x,y) = \begin{cases} 0, y = 2k, \\ x, y = 2k+1. \end{cases}$

10) $f(x,y) = \begin{cases} 0, y = 0, \\ x + y - 1, y > 0. \end{cases}$ Вернуться в **Задачи**

Глава 9. МАШИНЫ ТЬЮРИНГА 1. 1) $1^2 q_0 1^3$. 2) $q_0 101^3$. 3) $1 q_0 101$. 4) $101^4 q_0 0$.

5) $10 q_0 1^2$. 6) $1^2 q_0 1^2$. 7) $1^2 0 q_0 1^2$. 8) $1^2 0^4 q_0 1$. 9) $q_0 0^3 10^2 1$.

11) $10 q_0 01^2$. 2. 1) а) $T(P) = 1^3 01$; б) $T(P) = 101$. 2) а) Неприменима;

б) $T(P) = 1$. 3) а); б) Неприменима. 4) а) $T(P) = 1^2 01^3$; б) Неприменима. 5) а) Неприменима;

б) $T(P) = 1$. 6. 1) а) $1^4 01$; б) 1^5. 2) а) $1^2 01$;

б) 101^2. 3) а) 1^4; б) $1^2 01$. 7. 1) а) 1^2; б) 1. 2) а)1; б) 1. Вернуться в **Задачи**

Глава 10. ЭФФЕКТИВНОСТЬ АЛГОРИТМОВ

Сложность вычислений и эффективность алгоритмов составляют одну из важнейших проблем современной теории вычислительных систем. В области теории и практики программирования выработано много различных подходов к проблеме сложности вычислений. Задача настоящей главы – познакомить читателя с основами современных точных методов оценки сложности задач и эффективности алгоритмов. Изучение таких методов полезно для развития интуитивных представлений об эффективном (с точки зрения стоимости) использовании вычислительных машин и для

выработки практических навыков оценки потребности в вычислительных ресурсах (таких, как память и время), необходимых при наиболее благоприятных условиях для вычисления функции данной сложности на универсальных вычислительных машинах.

Переборные задачи и сложность вычислений

Задача распознавания П – это множество Zп всевозможных индивидуальных задач и подмножество Zп.да ⊂ Zп задач с ответом «да». Задача распознавания

П называется переборной, если каждая индивидуальная задача z формулируется следующим образом: существует ли такой объект y, что выполняется свойство, выражаемое предикатом R(z,y)? При этом предполагается, что проверка истинности R(z,y) имеет полиномиальную сложность. Например, задача 3-выполнимости к.н.ф.: дано множество дизъюнкций D={d1,d2,…,dm} на конечном множестве булевых переменных X={x0,x1,…,xn}, таких, что число |di| переменных каждой дизъюнкции равно 3. Требуется ответить на вопрос: существует ли на X набор значений истинности, при котором выполняются все дизъюнкции из D?

Предполагается, что для представления исходных данных используется алфавит Σ и некоторый естественный способ кодирования K, причем длина кода исходных данных задачи z равна l(z). Введем обозначение Σ для множества всевозможных цепочек символов (слов) в алфавите Σ. Любое подмножество множества Σ цепочек называется языком над алфавитом Σ. Множество текстов задачи П с ответом из множества Zп.да при выбранном способе кодирования K будем рассматривать как язык L(П,K).

Классы задач P и NP

Детерминированная машина M называется полиномиальной, если существует полином p такой, что для всех n∈N, TM ≤p(n). Класс P-языков определяется следующим образом: существует полиномиальная машина M, □

P =☐ L ☐. ☐ для которой L = LM ☐ Задача распознавания П при выбранном способе кодирования K принадлежит классу P, если L(П,K)∈P.

Недетерминированные вычисления

Рассмотрим недетерминированную машину Тьюринга (НДМТ) Му, которая для любой формулировки z∈ZП задачи распознавания П выдает следующий результат: если z∈ZП.да, то машина угадывает значение y, удовлетворяющее R(z,y), и записывает код y на ленту рядом с z; если z∉ZП.да, машина Му сообщает об этом. Обратите внимание, что машина Му не читает ленту, а только угадывает ответ и пишет его на ленту. Детерминированная машина Мпр по входу (z,y) проверяет истинность R(z,y). Рассмотрим НДМТ М в виде суперпозиции Му и Мпр. Эта машина решает задачу П за полиномиальное время, если найдется такой полином p, что для любой задачи z∈ZП.да машина Му найдет такое значение y, что детерминированная машина Мпр по значению (z,y) проверит истинность R(z,y) за время p(l(z)). Это означает, что размер y ограничен полиномом от l(z).

Класс NP – это все задачи распознавания, которые (при разумном кодировании) могут быть решены недетерминированным алгоритмом (N – non-deterministic) за полиномиальное время (P).

Для распознавания свойства R(z,y) можно сформировать пару задач.

- Прямая задача: верно ли, что для заданного z существует такое y, что выполняется R(z,y)?

- Обратная задача: верно ли, что для заданного z не существует такое y, что выполняется R(z,y)?

Если прямая задача принадлежит классу P, то и дополнительная задача принадлежит классу P. Если же прямая задача принадлежит классу NP, то не известно, принадлежит ли дополнительная задача классу NP.

Недетерминированная машина M принимает x, если, по крайней мере, одно вычисление для x является принимающим. Язык, распознаваемый программой M: LM={x∈Σ □M принимает x}.

Время, за которое принимается x∈LM, – это минимальное число шагов по всем вычислениям при входе x.

Временная сложность программы НДМТ M – это функция TM: N→N (N − множество целых чисел): существует такое значение x ∈ LM , x = n, TM (n) = max□ { } ∪ □m, что время принятия x программой машины M равно m TM(n)=1, если нет ни одного входа длины m, принимаемого программой M.

НДМТ имеет полиномиальную временную сложность, если найдется такой полином p, что TM(n)≤p(n) для всех n≥1.

Формальное определение класса NP: □ существует НДМТ M с полиномиальным NP = □ L временем работы, такая, что L = LM Хотя недетерминированный алгоритм угадывает y, зависящий некоторым образом от z, угадывающий модуль My при этом полностью игнорирует вход z.

Теорема. Если П∈NP, то существует такой полином p, что П может быть решена детерминированным алгоритмом с p(n) временной сложностью O (2).

Класс NP-полных задач

Полиномиальная сводимость NP-полных задач Если P≠NP, то между P и NPC=NP\P NP имеется существенное различие. Многолетняя мировая практика разработки алгоритмов показывает, что P еще никому не удалось построить алгоритм с полиномиальной временной сложностью для некоторых классов задач. Цель теории NP-полных задач в доказательстве результатов вида: Если P≠NP, то П∈ NP\P. Важную роль в теории NP-полных задач играет полиномиальная сводимость. Язык L1⊆Σ1 полиномиально сводится к языку L2⊆Σ2 , если существует функция f: Σ1 →Σ2 , удовлетворяющая двум условиям. 1. Существует ДМТ-программа, вычисляющая f с полиномиальной временной сложностью. 2. Для любого x∈Σ1 , x∈L1 в том и только том случае, если f(x)∈L2. Будем говорить «L1 сводится к L2» (опуская слово «полиномиально»)

и писать L1 ∝ L2. Утверждение 5.1. Если L1 ∝ L2, то из L2∈P следует, что L1∈P. Если П1 и П2 задачи распознавания, а K1 и K2 их схемы кодирования, то будем писать П1 ∝ П2, если L1(П1,K1) ∝L2(П2,K2).

Утверждение. Если L1 ∝ L2 и L2 ∝ L3, то L1 ∝ L3 (транзитивность). Доказательство можно найти, например, в [3].

Языки L1 и L2 полиномиально эквивалентны, если L1 ∝ L2 и L2 ∝ L1. Язык L называется NP-полным (L∈NPC), если L∈NP и любой другой L′∈NP сводится к L.

Языки L1 и L2 полиномиально эквивалентны, если L1 ∝ L2 и L2 ∝ L1. Язык L называ-

ется NP-полным (L∈NPC), если L∈NP и любой другой L′∈NP сводится к L.

Утверждение. Если L1 и L2 ∈NP, L1∈NPC и L1 ∝ L2, то L2∈NPC. Доказательство

можно найти, например, в [3].

Теорема. Задача о выполнимости к.н.ф. есть NP-полная задача.

Доказательство основано на описании работы машин Тьюринга с помощью к.н.ф..

Труднорешаемые задачи

NP-полная задача П называется универсальной, если к ней полиномиально сводится любая NP-полная задача. Выше была доказана принадлежность задачи выполнимости к.н.ф. к классу NP-полных задач.

Приведем другие примеры задач этого класса: о полном подграфе, о вершинном покрытии, о сложности д.н.ф. частичной булевой функции и о трехмерном сочетании.

Задача о полном подграфе. Граф называется полным, если любые две его вершины соединены ребром. По заданному неориентированному графу G и числу k требуется определить, имеется ли в G полный подграф с k вершинами.

Задача о вершинном покрытии. Некоторое подмножество V′, содержащее k=□V′□ вершин, образует вершинное покрытие мощности k графа G, если для любого ребра найдется инцидентная ему вершина в подмножестве V′.

По заданному графу G и числу k требуется определить, имеет ли G вершинное покрытие мощности k.

Задача о сложности д.н.ф. частичной булевой функции. По частичной булевой функции f и числу k необходимо установить, возможно ли построить д.н.ф. для f, содержащую не более k букв.

Трехмерное сочетание. Дано множество $M \subseteq W \times X \times Y$, где W,X,Y – непересекающиеся множества, содержащие одинаковое число элементов q. Необходимо установить, что M содержит трехмерное сочетание, т.е. подмножество $M' \subseteq M$ такое, что $\square M' \square = q$ и никакие два разных элемента M' не имеют ни одной равной координаты.

Например, W={a,b,c}, X={d,e,f}, Y={j,h,i} и M = {(a,d,i), (a,e,i), (a,f,j), (b,e,i), (b,f,j), (c,e,h), (c,d,i)}. Данный пример имеет положительное решение: $M' = \{(a,d,i), (b,f,j), (c,e,h)\} \subseteq M$.

Пусть П1 – задача о выполнимости к.н.ф. Тогда, чтобы доказать, что задача Пq является NP-полной, достаточно доказать, что $П1 \propto П2 \propto \ldots \propto Пq$.

Например, в [15] показано, что задача выполнимости к.н.ф сводится к задаче о полном подграфе, которая в свою очередь сводится к задаче о вершинном покрытии, а последняя – к задаче о сложности д.н.ф. частичной булевой функции.

NP-трудные задачи

Задача П называется NP-трудной, если П – произвольная задача (т.е. не обязательно, что $П \in NP$) распознавания свойств, к которой сводится некоторая NP-полная задача. К NP-трудным могут относиться не только задачи распознавания свойств, но и другие переборные задачи.

Точное решение NP-трудной (так же как и NP-полной) задачи в общем случае может быть получено только за экспоненциальное время. Следовательно, решать ее переборным алгоритмом неэффективно при большом количестве вариантов перебора.

ЛИТЕРАТУРА

1. *Бочкарева О.В.* Учебное пособие по математике (специальные главы). – М.: Радио и связь, 2001.
2. *Гаврилов Г.П., Сапоженко А.А.* Задачи и упражнения по курсу дискретной математики. – М.: Наука, 1992.
3. *Горбатов В.А.* Фундаментальные основы дискретной математики. – М.: Наука, 2000.
4. *Кузнецов О.П., Адельсон-Вельский Г.М.* Дискретная математика для инженеров. – М.: Энергоатомиздат, 1988.
5. *Кук Д., Бейз Г.* Компьютерная математика. – М.: Наука, 1990.
6. *Лавров И.А., Максимова Л.Л.* Задачи по теории множеств, математической логике и теории алгоритмов. – М.: ФИЗМАТЛИТ, 2001.
7. *Логинов Б.М.* Введение в дискретную математику. – Калуга, 1998.
8. Лихтарников Л.М., Сукачева Т.Г. Математическая логика: Курс лекций. Задачник-практикум и решения. – СПб.: Лань, 1999.
9. Математическая энциклопедия. Т. 1. – М.: Советская Энциклопедия, 1977.
10. *Мендельсон Э.* Введение в математическую логику. – М.: Наука, 1984.
11. *Непейвода Н.Н.* Прикладная логика. – Новосибирск: Изд-во Новосибирского университета, 2000.
12. *Новиков Ф.А.* Дискретная математика для программистов. – СПб.: Питер, 2000.

13. *Тишин В.В*. Теория алгоритмов, предикаты. – Самара, 2001.
14. *Фролов И.С*. Элементы математической логики. – Самара: Самарский университет, 2001.
15. *Яблонский С.В*. Введение в дискретную математику. – М.: Высшая школа, 2001.

В.И. Новоселов

МАТЕМАТИЧЕСКАЯ ЛОГИКА
И ТЕОРИЯ АЛГОРИТМОВ

Учебно-методическое пособие

Редактор *В.В. Мокрынина*
Компьютерная верстка *Ю.А. Ларичевой*

Подписано в печать 28.04.11. Формат 60x84 1/8
Офсетная печать. Объем 8,75 п. л.
Тираж 100 экз. Заказ 42.

Отпечатано в типографии КРСУ
720048, г. Бишкек, ул. Горького, 2